University of Bergen - Department of Informatics

Patient Reallocation and Waitlist Reduction in the Norwegian GP System

Author: Lars Møen Haukland

Supervisor: Fredrik Manne

UNIVERSITY OF BERGEN
Faculty of Science and Technology

June, 2026

Abstract

Many assignment problems share a common structure: each agent already holds one item, such as a house, a school place, or a doctor, and some agents would prefer to hold an item currently held by someone else. When no spare items are available, the only way to satisfy these agents is to find cycles of agents who can exchange their items simultaneously. This thesis studies how to find such exchanges, motivated by the allocation of general practitioners (GPs) in Norway, where people wanting to switch GP must wait for a slot to open because almost all GP lists are full. We formalise the problem as one of finding cycles in a directed graph of patients and GPs, and define three notions of a good set of exchanges, one that resolves as many patients as possible, one that gives strict priority to those who have waited the longest, and one that balances the two. We develop exact algorithms based on cycle cancelling for each of these, together with a fast heuristic, and compare them against an existing mechanism in a simulation of the Norwegian GP system. We find that the choice of objective has little effect on how many patients are resolved, but a large effect on which patients are resolved and therefore on how long they wait, and that the algorithms differ widely in running time.

Acknowledgements

I would like to thank my supervisor Fredrik Manne for his guidance, support and patience throughout this project. I would also like to thank Juni Weisteen Bjerde for new perspectives on the topics we discussed. In addition I thank Mathias Berntert for his input on the GP problem and on cycle cancelling in particular. My thanks also go to Ingrid Huitfeldt, Victoria Marone and Daniel Waldinger, whose work developing and testing their Top Trading Cycles mechanism inspired us to explore this problem. Their sharing of the replication package also made it possible to compare our algorithms against theirs, which was invaluable for my thesis. I am grateful to Tommy Odland for taking the time to help me understand the Top Trading Cycles mechanism and other related mechanisms.

I am also very appreciative of my fellow students, who made my degree not only more insightful but also fun and memorable. Ljubo, Jacob and Sindre in particular made my time in the reading hall far more enjoyable, with fun and interesting conversations, long breaks, and intense rounds of table tennis that finally helped me surpass my father's skill at the game.

I thank my family for always supporting and comforting me, and for making sure I did not forget I had a thesis to work on.

Lastly I would like to thank my girlfriend, Kristine, for her patience, love and support. Sometimes coming home to a warm dinner after being confused all day was just what I needed.

Contents

1. Introduction

The Norwegian general practitioner (GP) system, launched in 2001, aims to ensure the Norwegian population has a stable GP. A GP can follow their patients over time, creating a trusting and safe relationship [1]. Today, this system works as intended, and GPs handle most of the population's needs. As of December 2025, there are 5720 GPs in Norway serving a population of 5.6 million. The number of patients each GP has varies, but on average each GP has about 1,000 patients [2].

One inefficiency in the Norwegian GP system is when a person wants to switch from one GP to another. If the GP they want to switch to does not have any free capacity for new patients, the person has to sign up in a queue and wait for a free slot to open up. In December 2025, the number of open GP lists was 1340, indicating that 77% of all GPs had no extra capacity [3]. In a more densely populated city such as Bergen, the numbers were even worse. Only 7 out of 268 GPs had available capacity for new patients. Thus, 97% of the GPs in Bergen had no extra capacity [3].

When a person wants to switch to a GP with no excess capacity, they must sign up and wait in line. Open slots are allocated on a first-come, first-serve basis. In December 2025, about 300,000 people were waiting to see a GP in Norway [2]. As there were only 5720 GPs, each had an average of more than 50 people on their waiting list Since there is a relatively large number of people who are waiting to either switch or be allocated a GP, it follows that it is likely that there are cycles, in that a person P_A wants GP D_A, who currently has P_B as a patient, but P_B wants to switch to GP D_B, who currently has P_A as a patient. In this case, patient P_A ends up waiting for an open slot with P_B's GP, and vice versa. Logically, instead of waiting, P_A and P_B could just swap GPs. This can be generalized to longer "waiting cycles" that could all be serviced by allowing for simultaneous switching.

In this thesis, we study how such cycles can be resolved and propose different strategies for resolving these. In particular we investigate the use of cycle detecting algorithms to find optimal switches between patients and show that this can lead to a substantial reduction in the number of people on waiting lists by running these algorithms periodically. While the focus is always on helping as many patients as possible we study two different variations of this problem, when there is a priority among those waiting and when there is no priority. We have developed and tested optimal algorithms as well as heuristics for these cases and evaluated them for solution quality as well as speed.

In addition we compare our result with those of the existing algorithms. We note that our solutions have applications to other types of reallocation problems where users want to switch their given assignment. This could for instance be in reallocation of houses, kidney donors or students switching schools.

1.1. Thesis Outline

The remainder of this thesis is organized as follows:

- **Section 2** provides background on the Top Trading Cycles mechanism, cycle cancelling algorithms and how these methods have been used on similar problems.

- **Section 3** formally defines the GP allocation problem and three notions of an optimal solution that differ in how they prioritise patients.
- **Section 4** describes the algorithmic strategies employed in our implementations.
- **Section 5** presents the experimental results, comparing the algorithms on how many patients they resolve, how long patients wait, and how fast they run.
- **Section 6** concludes and discusses future work.

2. Background

Problems like the GP allocation problem have been studied for quite some time with a number of variations. In addition there are different perspectives on how to solve these problems. For instance in economics the focus is often on the fairness of an assignment, trying to optain a stable and strategy proof assignment. In this chapter we describe similar problems to the GP allocation problem, describe the Top Trading Cycles algorithm (TTC) and the use of this by Huitfeldt et al for the GP allocation problem [4]. Finally we describe cycle cancelling, an optimization technique that our exact algorithms are based on.

2.1. Related problems

There are many different variations of assignment problems. In a typical setting there are agents and objects where the agents want to obtain particular objects. In the GP allocation problem the patients are the agents and GPs are the objects. The typical assignment problems are one-to-one meaning that one agent is assigned to at most one object and an object can only be "owned" by one agent. In addition we also have many-to-one assignment problems, like the GP allocation problem where one agent can only be assigned to one object but one object can be "owned" by more than one agent. The terminology that a patient "owns" a GP means that the patient has that GP as his or hers current GP. It might be misleading to say that our patients "own" GPs but this makes more sense in other problems that we will describe, owning a GP in our case means having that GP as a current GP.

2.1.1. One-to-one assignment problems

Recall that in a one-to-one assignment problem each agent is assigned to at most one object and each object can be "owned" by at most one agent. We look at two such problems, the Housing Marked and the Kidney Exchange. Both differ from the GP allocation problem since a GP can be owned by many patients at once, but can be solved using the same algorithms.

2.1.1.1. Housing Market

The Housing Market model introduced in Shapley and Scarf [5] describes a model with traders (agents) and indivisible goods (objects). These goods can be houses, making it a housing market. Each agent already has one house but may want to switch. Every agent has a preference list ordering every house in order of preference, Shapley and Scarf combine all these preference lists to make a preference matrix [5].

It is in this article from Shapley and Scarf [5] that they also introduce the Top Trading Cycles algorithm and describe how it is fitting for models like the Housing Market. We also see similarities to the GP allocation problem in that we have agents already owning an object but wanting to switch. However two differences are that a GP can have multiple patients while a house can only be owned by one agent, and also that in the Housing market each agent has a complete preference list of objects while in the GP assignment problem each agent has only one preferred GP.

2.1.1.2. Kidney exchange

The Kidney Exchange problem describes an issue when trying to find compatible kidney donors. It describes the case where we have pairs of patients and donors, but the patient is

incompatible with its donor's kidney. We then need to find some way to swap around the donors to compatible patients. Either with pairwise swapping or through longer cycles. If we visualize the problem as a directed graph, each vertex is a patient and donor pair. An edge from pair A to pair B means that donor A is compatible with patient B. Now cycles can form as in Figure 1 where we have a cycle of length three $A \to B \to C$. This means that the donor in pair A can be matched with the patient in pair B and so on, assuring that each patient in the cycle is assigned a compatible donor.

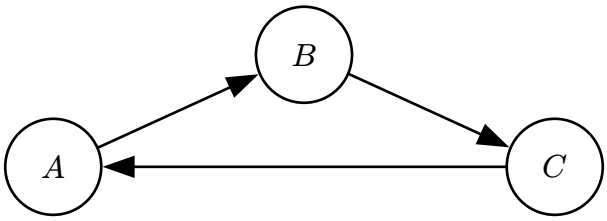

Figure 1: An example graph for the kidney exchange problem.

Abraham, Blum and Sandholm [6] show that, with unbounded cycle length, a maximum-weight exchange can be found in polynomial time [7]. An issue often arising with unrestricted cycle length is if a donor suddenly refuses to donate. If this happens after their patient has received a kidney then a patient is left without a donor and cannot exchange later. Because of this the problem is often considered with a restricted cycle length to allow all operations to be executed at the same time. This way no donor can refuse in the middle. For each pair in a cycle you need 2 operations, for a cycle of length k you need $2k$ operations at the same time. It is for this reason that the Kidney Exchange problem often is considered with a max cycle length k, as the number of operating rooms in a state/city is limited. Finding a maximum-cardinality exchange with cycles of length at most k, for any fixed $k \geq 3$, is an NP-hard computational problem [6], [7].

The Kidney Exchange problem and the GP allocation problem have much in common. In both problems agents already have an object, but want to switch for another. We need to find cycles to exchange the objects in such a manner that as many patients as possible are satisfied. The key difference is that only one donor can be "owned" by one patient while in the GP allocation problem a GP can be "owned" by multiple patients.

2.1.2. Many-to-one assignment problems

In a many-to-one assignment problem each agent is still assigned to at most one object, but an object can be "owned" by more than one agent. This is the same structure as the GP allocation problem, where a GP can have many patients. We look at one such problem, the College Admissions problem.

2.1.2.1. The College Admissions problem

The College Admissions problem, introduced by Gale and Shapley [8], describes the problem of assigning applicants to colleges. Each college has a certain quota, i.e. a maximum number of applicants that can be admitted and each applicant ranks the colleges in order of preference. An applicant does not have to rank every college and can skip colleges he or she does not want to attend [8].

The problem is then to find a stable assignment, meaning that there does not exist a case where there are two applicants α and β who are assigned to colleges A and B, although β prefers A to B and A prefers β to α. Such a pair would break the assignment, since both would rather be matched to each other.

There can be many stable assignments for the same preferences, so there is not one single best assignment. Among all stable assignments there is one that is best for every applicant at the same time, and one that is best for every college at the same time. Gale and Shapley [8] introduce an algorithm called deferred-acceptance that finds a stable assignment in polynomial time [8]. The algorithm has one side propose and the other side accept or reject. The side that proposes ends up with the assignment that is best for them and worst for the other side.

There are some similarities between the College Admissions problem and the GP assignment problem. Like how applicants prefer some colleges while patients prefer a GP. One key difference is that patients, only prefer one GP. Another difference is that patients already have existing GPs, while applicants do not already have a college they want to switch from.

2.2. Top Trading Cycles

The Top Trading Cycles, or TTC, is an algorithm that finds cycles of agents that can all trade their objects at the same time. We look at it in two forms. First we explain the classical TTC, which works on the Housing Market problem. This is the original version and we explain it to show where the method comes from. Then we explain the version made by Huitfeldt et al. for the GP allocation problem. It is this version that inspired our work, and it is the one we later compare our algorithms against.

2.2.1. Classical TTC

The Top Trading Cycles algorithm (TTC), developed by Gale and published by Shapley and Scarf [5], is an algorithm to find a re-allocation of objects, or goods, without using money. It was introduced as a way of solving the Housing Market problem. The algorithm finds a re-allocation of houses to traders, such that all mutually-beneficial exchanges have been realized [5].

TTC works on a directed graph. For the Housing Market problem each agent is a node. For an agent i we let $\mathrm{Top}(i)$ denote the agent holding the house that i prefers the most among the houses still in the graph, if i's own house is the most preferred then $\mathrm{Top}(i) = i$. The algorithm is as follows [5]:

1. Insert a directed edge from each agent i to $\mathrm{Top}(i)$.
2. Find a cycle, which is guaranteed to exist since every agent has exactly one outgoing edge, and execute the trades in that cycle. Then remove all involved agents from the graph.
3. If there are remaining agents, repeat from step 1.

This guarantee is the pigeonhole principle, a finite graph where every node has exactly one outgoing edge must contain a cycle. The cycle can be of length 1, when an agent's top house is its own, then the agent simply keeps their house and is removed. For $\mathrm{Top}(i)$ to always be defined the agent does not need to rank every house. It is enough that each agent has a strict ranking of some of the houses that includes their own house. An agent's own house stays in the graph as long as the agent does, so $\mathrm{Top}(i)$ always finds a house no further down the list

than the agent's own. Houses ranked below the agent's own house can never be assigned to them, so they can be left out entirely.

We now consider an example with three agents, A, B and C, owning houses h_A, h_B and h_C, each with a strict preference list over the houses:

- $A = [h_B, h_C, h_A]$
- $B = [h_C, h_A, h_B]$
- $C = [h_B, h_C, h_A]$

If we make the graph where each agent points to their Top we get Figure 2.

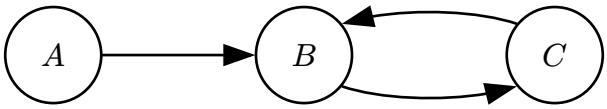

Figure 2: An example Housing Market for TTC.

We have the cycle $B \to C \to B$, shown in Figure 2, so we execute the trades, B gets h_C and C gets h_B, and remove both agents from the graph. The last remaining agent is A, and as h_B and h_C are no longer in the graph, $\text{Top}(A)$ is now A itself, shown in Figure 3.

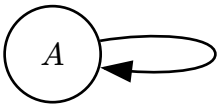

Figure 3: An example Housing Market for TTC, A's Top is itself.

We execute this trivial trade, A keeps h_A, and are left with no more agents, making TTC terminate.

Roth proved that TTC is strategy proof, meaning all agents are motivated to report their true preferences [9]. TTC also satisfies individual rationality, that an agent is at least as well off by participating, and Pareto efficiency, that the objects are allocated such that no agent can improve without worsening another agent [5], [9].

Now using TTC for the GP assignment problem is challenging, but not because patients rank only two GPs. Note that a patient's preferences are exactly a list of the minimal form above, the preferred GP followed by their current GP. The real difference is that several patients can hold the GP that patient i prefers, so $\text{Top}(i)$ is no longer a single agent. How should we choose between these? This is what Huitfeldt et al. [4] address. Notably they show that in their dynamic setting, where the mechanism is run repeatedly and patients care about waiting time, TTC loses some of the classical properties, it is no longer strategy proof and some patients can be left worse off than under the existing first come first served system. Later we run their TTC for the GP assignment problem and compare it with our algorithms, but first we explain it and how it differs from the classical TTC.

2.2.2. GP assignment problem TTC

In their paper, Huitfeldt et al. [4] study the GP assignment problem using real patient and GP data from Norway. Before explaining their algorithm we need one piece of terminology. The set of patients currently enrolled with a GP is called the GP's *panel*, and each GP has a cap on

how many patients their panel can hold. A GP has available capacity when their panel holds fewer patients than its cap, such a GP has open slots that a waiting patient can fill directly, without any exchange.

In their model some GPs have available capacity. Their mechanism therefore first runs a waitlist algorithm that fills all open slots, it goes through every GP with available capacity and assigns the waiting patients with the highest priority to the open slots, until no patient is waiting for a GP with available capacity [4]. Only after this do they run TTC on the patients that remain on waitlists. In our setting every GP is at full capacity, so the waitlist step never assigns anyone, and in the following we explain their TTC without it.

The algorithm works on a graph with both the waiting patients and their GPs as nodes:

1. Each patient points to their preferred GP, if the preferred GP has been removed from the graph the patient points to their current GP instead. Each GP has a preference list consisting of its panel members in a fixed arbitrary order, followed by the waiting patients wanting that GP in priority order. The GP points to the first patient in this list that is still in the graph.
2. Find a cycle in the graph and resolve it, every patient in the cycle is moved to the GP they point to. Remove the resolved patients from the graph. If a GP has no panel members left in the graph, remove the GP.
3. Repeat from step 1 until there are no more patients.

Like in the classical TTC algorithm a cycle is guaranteed to exist by the pigeonhole principle, every node in the graph, patient or GP, has exactly one outgoing edge. A cycle can also consist of just a patient and their own GP pointing at each other, then the patient simply stays with their current GP and is removed. Huitfeldt et al. also propose a variant with adjusted priorities, TTC with priority, that protects patients whose GPs have available capacity, since in our setting no GP has available capacity we compare only against the basic TTC [4].

2.3. Cycle cancelling

Cycle cancelling is a technique used in flow and circulation problems. In these problems each edge of a graph has a cost, and the goal is to find a circulation of flow whose total cost is as small as possible. First we define the terms needed for cycle cancelling, then we present the technique itself. We use definitions from Ahuja, Magnanti and Orlin [10].

2.3.1. The minimum cost circulation problem

Let $G = (V, E)$ be a directed multigraph with n vertices and m edges. Each edge is a triple (v, w, i) where v is the tail, w is the head and i is an index. The pair (v, w) gives the endpoints of the edge. The index i separates edges that have the same endpoints, so G can have parallel edges. This means an edge is identified by the full triple and not just by its endpoints.

The multigraph G is a circulation network if each edge (v, w, i) has a capacity $u(v, w, i) \geq 0$ and a cost $c(v, w, i) \in \mathbb{Z}$. For a vertex $w \in V$ we let $\text{in}(w) = \{(v, w, i) \in E\}$ be the edges whose head is w, and $\text{out}(w) = \{(w, v, i) \in E\}$ the edges whose tail is w.

Definition 2.3.1.1 (Circulation): A circulation is a function f that assigns a value $f(v, w, i)$ to each edge and satisfies the capacity constraints

$$0 \leq f(v, w, i) \leq u(v, w, i) \qquad \forall (v, w, i) \in E \tag{1}$$

and the conservation constraints

$$\sum_{(u,w,i) \in \text{ in}(w)} f(u, w, i) = \sum_{(w,v,j) \in \text{ out}(w)} f(w, v, j) \qquad \forall w \in V. \tag{2}$$

The conservation constraint states that at each vertex the flow coming in equals the flow going out. No flow enters or leaves G from the outside, so all flow circulates in the network. The cost of a circulation f is

$$\text{cost}(f) = \sum_{(v,w,i) \in E} c(v, w, i) f(v, w, i). \tag{3}$$

Definition 2.3.1.2 (Minimum cost circulation problem): Given a circulation network G, the minimum cost circulation problem is to find a circulation f with minimum cost.

Note that the costs are what make the problem interesting. If every edge has a cost $c(v, w, i) \geq 0$, then the circulation with zero flow on every edge always has minimum cost, and the problem is trivial. The problem becomes meaningful when some edges have negative costs. Then the zero circulation is still feasible, but pushing flow along negative cost edges lowers the cost, so the problem becomes to find where flow should be pushed. In Section 4 we show how to construct circulation networks for the GP assignment problem, where each edge is a possible patient switch and all costs are negative, so that a minimum cost circulation corresponds to a best set of switches. For example, if every edge has cost -1, then $\text{cost}(f) = -\sum_{(v,w,i) \in E} f(v, w, i)$, the negative of the total flow, so a circulation with minimum cost is equivalent to a circulation with the most flow.

An example circulation network is shown in Figure 4. For simplicity this example is not a multigraph. In the example each edge is labeled x, y, meaning the edge has cost x and capacity y, and every edge has cost -1 and capacity 1. The network has three feasible circulations: no flow on any edge, the cycle $d \to c \to b \to d$, and the cycle $a \to b \to d \to c \to a$. Their costs are 0, -3 and -4. The longer cycle is the optimal circulation since it has the most flow and the lowest cost.

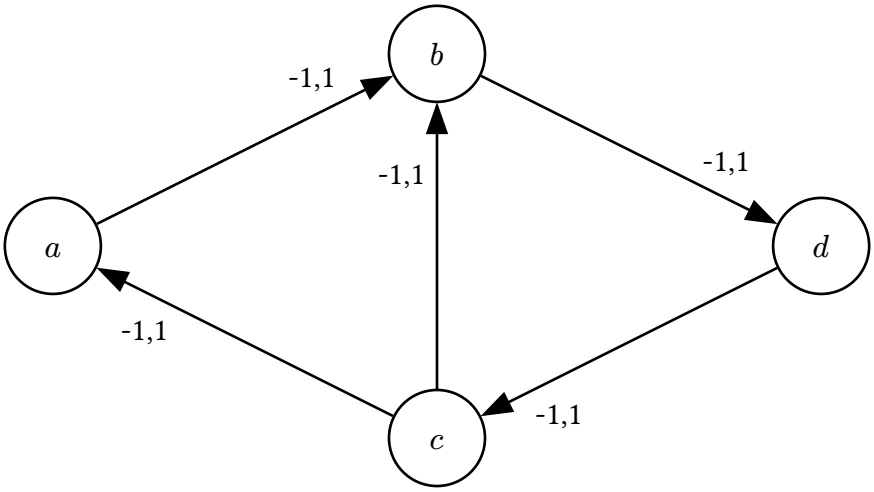

Figure 4: An example circulation network.

2.3.2. The residual network

The residual network shows what capacity is left after a circulation has been computed, and what flow can be undone. Given a circulation network G and a circulation f, we build the residual network $G(f)$ edge by edge. Each edge $(v, w, i) \in E$ gives two residual edges.

- A forward edge (v, w, i) with cost $c(v, w, i)$ and residual capacity $u_f(v, w, i) = u(v, w, i) - f(v, w, i)$.
- A backward edge (w, v, i) with cost $-c(v, w, i)$ and residual capacity $u_f(w, v, i) = f(v, w, i)$.

A residual edge is in $G(f)$ only if its residual capacity is greater than 0. An edge with residual capacity 0 is saturated. We build the residual edges from each edge (v, w, i) and not from the pair (v, w), so the index i is kept on both residual edges. This means that $G(f)$ is also a directed multigraph and every residual edge is still uniquely identified.

The two kinds of residual edges have different roles. A forward edge has the same cost as its original edge, and pushing more flow through it increases f on that edge. A backward edge has the negated cost, and pushing flow through it decreases f on the original edge, so a backward edge undoes earlier flow. A backward edge is in $G(f)$ only as long as there is positive flow on its original edge. This means that we can only undo flow that is actually there.

A negative cost cycle in $G(f)$ is a directed cycle of residual edges where the costs sum to a negative number. The following theorem from Ahuja, Magnanti and Orlin [10] states when a circulation is optimal.

Theorem 2.3.1 (Negative Cycle Optimality Theorem): A feasible circulation f^* is an optimal solution of the minimum cost circulation problem if and only if the residual network $G(f^*)$ contains no negative cost directed cycle.

At first glance the theorem may seem to say that any circulation carrying flow is suboptimal, since pushing flow creates backward edges with positive cost. This is not the case. If we push

flow around a cycle where every edge has a cost of -1, we do create backward edges of cost $+1$, but these form a cycle with positive cost, not a negative one. So a circulation can carry flow and still have no negative cycle in its residual network. A negative cost cycle in $G(f)$ instead represents an improvement that is still possible. Such a cycle can use forward edges, where we push new flow, and backward edges, where we reroute flow we already committed. When no negative cycle is left, no improvement is left, and by Theorem 2.3.1 f is optimal.

2.3.3. Algorithm

Using the Negative Cycle Optimality Theorem we can now formalize the cycle cancelling algorithm. The idea is simple. We start with some feasible circulation, and while the residual network has a negative cost cycle we cancel that cycle. The theorem tells us that when there are no negative cost cycles left, the circulation is optimal.

The starting circulation can often be 0 on all edges. In general, a circulation problem can also have lower bounds on the flow of each edge, requiring some edges to carry a minimum amount of flow. Then the zero circulation is not feasible, and a feasible starting circulation must first be computed, which can be done with a max-flow computation on a modified network [10]. In the circulation networks we construct for the GP assignment problem all lower bounds will be 0, so the zero circulation is always feasible and we use it as the start. The outline of the algorithm is therefore as follows:

```
1 Start with any feasible circulation f, in our case f = 0
2 while ∃ negative residual cycle c
3 | Cancel c: push capacity(c) units of flow along every residual edge of c
```

Here $\text{capacity}(c)$ is the smallest residual capacity of any edge in c. What pushing flow means depends on the type of residual edge. Recall that a forward edge (v, w, i) comes from an edge with leftover capacity, and a backward edge (w, v, i) comes from an edge that already has flow on it in the reverse direction. When the cycle uses a forward edge (v, w, i) we increase the flow $f(v, w, i)$ on that edge. When the cycle uses a backward edge (w, v, i) we decrease the flow $f(v, w, i)$ on the original edge. This is how the algorithm can undo earlier flow.

After we cancel a cycle the residual network changes. An edge that we pushed flow on has less leftover capacity, and its forward edge can become saturated. At the same time its backward edge gets more residual capacity, since there is now more flow that can be undone. This is why a later cycle can push flow back using a backward edge and undo a forward flow made earlier.

2.3.4. Runtime

Finding the negative residual cycles can be done using the Bellman-Ford algorithm in $O(nm)$ time [10]. For each negative residual cycle that is cancelled the total cost of the circulation decreases by at least 1. The cost is bounded below by $-mCU$ where C is the maximum cost and U is the maximum capacity of any edge. It follows that the number of iterations is bounded by $O(mCU)$, and then the running time is $O(nm^2CU)$. Note that this running time is pseudo-polynomial, as it depends largely on the size of C and U.

Goldberg and Tarjan proved that a variant of the cycle cancelling algorithm called Minimum Mean Cycle Cancelling has a running time that is strongly polynomial [11]. For the GP assignment problem however, we will later show that the classical Cycle Cancelling algorithm is already polynomial.

3. Problem Formalization

In this chapter we define the GP allocation problem, what a feasible solution to it is, and three variants of what an optimal solution is. Note that in Section 2.3 the costs were placed on the edges of a graph, while in this chapter the priorities are placed on the patients. The two are connected, in Section 4 we construct graphs for the GP allocation problem where each patient corresponds to an edge, and the priority of the patient becomes the cost of that edge.

3.1. The GP allocation problem

3.1.1. Input

In the GP allocation problem we are given a set of patients $P = \{p_1, p_2, ..., p_n\}$ and a set of GPs $D = \{d_1, d_2, ..., d_m\}$. We are also given the current and preferred GP assignments for each patient:

- $D_{\text{cur}}[i]$ denotes the index of the current GP assigned to patient p_i (or $\perp$ if unassigned)
- $D_{\text{pref}}[i]$ denotes the index of the preferred GP for patient p_i

In addition we are given a priority function $R : P \to \mathbb{N}$, mapping some positive integer to each patient, with higher numbers indicating a higher priority to help this patient.

3.1.2. Feasible solution

Definition 3.1.2.1 (Feasible solution):

A subset $S \subseteq P$ of switching patients is a **feasible solution** if the patients in S can take over each other's slots such that every patient in S ends up at their preferred GP and the number of patients at each GP is unchanged. Formally, define the directed graph $G_S = (S, E_S)$ where

$$E_S = \{(a, b) \mid a, b \in S, \text{the preferred GP of } a \text{ is the current GP of } b\}. \quad (4)$$

An edge (a, b) means that patient a can take over the slot of patient b. Then S is feasible if and only if G_S contains a spanning subgraph that is a vertex-disjoint union of directed cycles covering every patient in S, that is, a selection of edges where every patient has exactly one outgoing and one incoming edge among the selected.

Note that a patient can have several outgoing edges in G_S, one to each patient in S that currently has their preferred GP. A solution consists of choosing one of them for each patient, such that the chosen edges form the cycles.

3.1.3. Optimization criterion

Next we define what it means for one feasible solution to be better than another. We might want a solution that exchanges as many patients as possible, a solution that exchanges the patients with the highest priority, or some mix of these. To compare solutions we use an ordering.

Definition 3.1.3.1 (Optimization criterion): An optimization criterion is an ordering $\succ$ over feasible solutions. A solution is optimal if no feasible solution is greater under $\succ$.

Each of the following variants of the GP allocation problem is defined by choosing such an ordering, and the goal of the algorithms in Section 4 is to find an optimal solution under the chosen ordering. We build on this to create our three main variants.

3.1.3.1. Lexicographic maximization by priority

Definition 3.1.3.1.1 (Characteristic vector): Let patients be ordered by priority: $p_1 \geq p_2 \geq ... \geq p_n$ where p_1 has the highest priority, with ties broken by a fixed arbitrary order, in our case the patient identifier. This makes the ordering a total order, so the position of each patient in the vector is well-defined. Given a feasible solution $S \subseteq P$, the **characteristic vector** is:

$$\chi(S) = (b_1, b_2, ..., b_n) \in \{0,1\}^n, \quad b_i = \begin{cases} 1 \text{ if } p_i \in S \\ 0 \text{ otherwise} \end{cases} \tag{5}$$

Definition 3.1.3.1.2 (Lexicographic ordering by priority):

$$S \underset{\text{lex}}{\succ} S' \quad \text{iff} \quad \chi(S) \text{ is lexicographically greater than } \chi(S') \tag{6}$$

That is, at the first index i where S and S' differ, $\chi(S)_i = 1$ and $\chi(S')_i = 0$.

Intuitively, $\chi(S)$ is a binary number whose most significant non-zero bit corresponds to the highest-priority patient that gets helped. The optimal solution is then the one that maximises this binary number: always satisfying the highest-priority patient it can, then subject to that the next highest, and so on.

Note that when priorities are tied, the tie-break order matters for which solution is optimal. Two tied patients occupy two distinct positions in the vector, and the one placed first wins any comparison between solutions that disagree on them, even if choosing the other would have allowed more patients in total to be satisfied. So the optimal solution under $\underset{\text{lex}}{\succ}$ is always defined relative to the chosen tie-break order, different tie-break orders can give optimal solutions of different sizes.

Example (Ordering two solutions lexicographically by priority):

We use the example graph in Figure 5 as the graph to create solutions from.

Given solutions
1. $S = \{p_4, p_5\}$ represents the subgraph G_S containing the cycle $p_4 \to p_5 \to p_4$
2. $S' = \{p_1, p_2, p_3\}$ represents the subgraph $G_{S'}$ containing the cycle $p_1 \to p_2 \to p_3 \to p_1$

For simplicity's sake we let $R(p_i) = i$, so p_5 has the highest priority.

Ordering all five patients by priority gives $p^{(1)} = p_5, p^{(2)} = p_4, ..., p^{(5)} = p_1$. Then:

$$\chi(S) = (1, 1, 0, 0, 0) \quad \chi(S') = (0, 0, 1, 1, 1) \tag{7}$$

At the first differing position ($i = 1$), $\chi(S)_1 = 1 > 0 = \chi(S')_1$, so $S \underset{\text{lex}}{\succ} S'$.

So for our first variant we want to find the optimal solution under the $\underset{\text{lex}}{\succ}$ ordering. This means always prioritizing patients with highest priority.

3.1.3.2. Maximizing cardinality

Another natural variant is to find a solution with the largest cardinality, e.g. a solution that contains the most patients.

Definition 3.1.3.2.1 (Ordering by cardinality):

$$S \underset{\text{size}}{\succ} S' \text{ iff } |S| > |S'| \tag{8}$$

An optimal solution will then be one that exchanges the most patients.

Example (Ordering two solutions by cardinality):

Using the same solutions as in the previous example:
1. $S = \{p_4, p_5\}$ represents the subgraph G_S containing the cycle $p_4 \to p_5 \to p_4$
2. $S' = \{p_1, p_2, p_3\}$ represents the subgraph $G_{S'}$ containing the cycle $p_1 \to p_2 \to p_3 \to p_1$

Under $\underset{\text{size}}{\succ}$: $|S| = 2$ and $|S'| = 3$, so $S' \underset{\text{size}}{\succ} S$. However the optimal solution is $\{p_1, p_2, p_3, p_4, p_5\}$, the union of both cycles.

Notice that the same two sets are reversibly ordered under the two criteria: $S \underset{\text{lex}}{\succ} S'$ (because S contains the highest-priority patient p_5) but $S' \underset{\text{size}}{\succ} S$ (because S' has more patients).

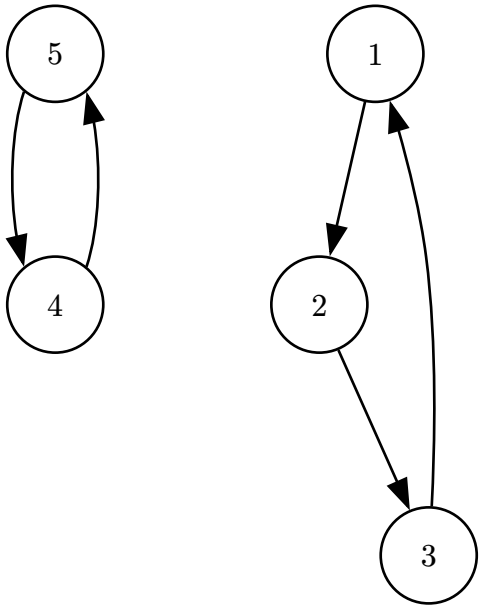

Figure 5: An example graph G.

3.1.3.3. Maximizing utility

Finally we formulate an ordering that is a combination of $\underset{\text{lex}}{\succ}$ and $\underset{\text{size}}{\succ}$. The lexicographic ordering can let a single high priority patient outweigh any number of lower priority patients, while the cardinality ordering ignores priorities completely. A natural middle ground is to let every patient count, weighted by their priority. We define the total utility to be the sum of the priorities of the patients in S.

We recall that $R : P \to \mathbb{N}$ is the function mapping a patient to a priority.

Definition 3.1.3.3.1 (Total utility function):

$$U(S) = \sum_{p \in S} R(p) \tag{9}$$

Definition 3.1.3.3.2 (Ordering by total utility):

$$S \underset{\text{util}}{\succ} S' \text{ iff } U(S) > U(S') \tag{10}$$

This means that even though solution S' contains some high priority patients, if S contains enough lower priority patients then S can still be a higher ranked solution under $\underset{\text{util}}{\succ}$.

Example (Ordering two solutions by total utility):

We use the following solutions from Figure 6:
1. $S = \{p_1, p_3, p_4\}$ representing the cycle $p_1 \to p_3 \to p_4 \to p_1$
2. $S' = \{p_5, p_2\}$ representing the cycle $p_5 \to p_2 \to p_5$

As in the previous examples we let $R(p_i) = i$. This gives the following total utilities:
$U(S) = 1 + 3 + 4 = 8$
$U(S') = 2 + 5 = 7$

It follows that $S \underset{\text{util}}{\succ} S'$. Here we see that the solution with more patients with lower maximum priority has a higher score than another with greater maximum priority.

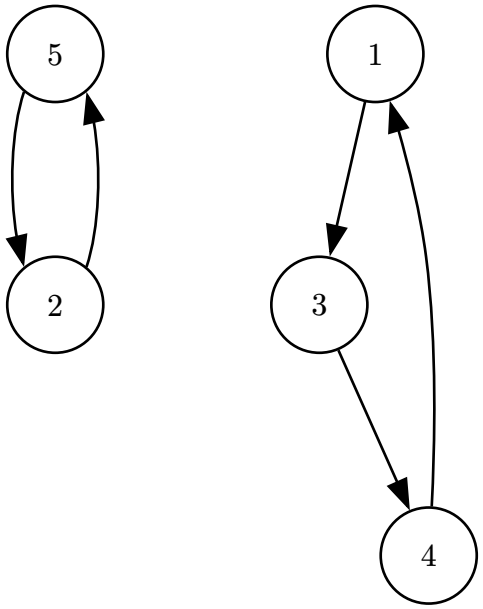

Figure 6: An example graph G.

3.1.4. Priority function

The priority function has quite a large effect on the final solution. The priority function chosen will largely decide which patients will be in the final solution. This is why it is important to investigate what effects the function can have on solutions when we are maximizing the ordering $\underset{\text{util}}{\succ}$ or $\underset{\text{lex}}{\succ}$.

If we make the priority function exponential, such as $R(a) = 2^a$, then the ordering $\underset{\text{util}}{\succ}$ becomes the same as $\underset{\text{lex}}{\succ}$. This is because 2^a is larger than the sum of all lower priorities below it. If every patient has a distinct priority, so that patient a has priority 2^a, then no group of patients with lower priority can have a total priority that reaches 2^a, since their priorities sum to at most $2^0 + 2^1 + ... + 2^{a-1} = 2^a - 1$. So a solution maximizing $\underset{\text{util}}{\succ}$ will always pick the higher priority patient first, which is exactly what $\underset{\text{lex}}{\succ}$ does. Note that this argument requires the priorities to be distinct, two patients that share the priority 2^a together outweigh one patient with priority 2^{a+1}.

While this ordering is in a sense the most fair, since we never let a patient with lower priority switch before one with greater, it might not always be the best for the collective good, since a high priority patient might block several lower priority patients from switching. One solution to this could be a priority function where $R(a)$ depends on how long patient a has been waiting for a switch. Then if we have the choice between patient p_i who has waited 30 days, $R(p_i) = 30$, and four patients who have waited 40 days in total, an algorithm maximizing $\underset{\text{util}}{\succ}$ will choose the four patients.

We can also use a priority function in between these two extremes. Instead of a fixed base we use a base k with $1 \leq k \leq 2$ and set

$$R(a) = k^{\text{days patient } a \text{ has been waiting}}. \tag{11}$$

The base k controls how much higher priority patients are favored. At $k = 1$ every patient gets priority 1, no matter how long they have waited. The total utility $U(S)$ is then just the number of patients in S, so maximizing $\succ_{\text{util}}$ becomes the same as maximizing $\succ_{\text{size}}$.

At $k = 2$ the priorities grow fast enough that a higher priority patient is worth more than any group of lower priority patients below them, so $\succ_{\text{util}}$ becomes the same as $\succ_{\text{lex}}$, as shown above. This equivalence only holds exactly when all waiting times are distinct. With this priority function many patients share the same number of days waited, so the priorities are not distinct and the argument above no longer applies. The lexicographic ordering itself is also only defined up to tie-breaking when priorities are equal, so with ties the $k = 2$ endpoint approximates $\succ_{\text{lex}}$ rather than matching it.

So the cardinality and lexicographic orderings are not separate cases but the two endpoints of the utility ordering. This is why our exact algorithm for $\succ_{\text{util}}$, which we present in Section 4, also solves $\succ_{\text{size}}$, by giving it the priority function with $k = 1$. The lexicographic case at $k = 2$ is different in practice, for two reasons. First, the priorities 2^a grow very large, and the runtime of cycle cancelling depends on the size of the costs, so running the utility algorithm with $k = 2$ would be slow. Second, as noted above, with tied waiting times the $k = 2$ endpoint does not match $\succ_{\text{lex}}$ exactly. For these reasons we treat strict lexicographic priority as its own case with its own algorithm, which we describe in Section 4. That algorithm does not encode priorities in weights at all, it processes patients in priority order with ties broken by a fixed order, and so handles ties exactly.

4. Implementation

In this chapter we present the algorithms we have implemented. We have implemented five algorithms in total. Three of them are exact algorithms based on cycle cancelling, *Cycle Cancelling for cardinality*, *Cycle Cancelling for utility* and *Cycle Cancelling for strict priority*, one for each of the orderings defined in Section 3. By exact we mean that the algorithm is guaranteed to return an optimal solution under its ordering. In addition we have implemented *Greedy DFS*, a fast heuristic, and the TTC of Huitfeldt et al. [4] that we compare against. For each algorithm we first motivate it, then describe how it works, and finally analyze its running time. For the exact algorithms we also give, or refer to existing, proofs of why they are exact, and show that they run in polynomial time for our priority functions.

4.1. Different graph representations used

The algorithms use different graph representations of the GP allocation problem. The reason is that the different orderings from Section 3 care about different aspects of the problem. For cardinality only the number of patients wanting each switch matters, so identical requests can be collapsed together. For the priority based orderings each patient must be kept apart, since each patient carries their own priority. The heuristic instead needs the patients and GPs as explicit nodes to search through. We therefore start by defining the three representations, for each we give the formal definition first and then explain it. Throughout, let $I = \{0, \ldots, |P| - 1\}$.

4.1.1. Patient and GP graph

$$\begin{gathered} G = (V, A), \quad V = P \cup D \\ A = \left\{ \left(p_i, D_{\text{pref}}[i]\right) \mid i \in I \right\} \cup \left\{ \left(D_{\text{cur}}[i], p_i\right) \mid i \in I \right\} \end{gathered} \tag{12}$$

The *Patient and GP graph* contains both patients and GPs as vertices. An edge $p_i \to D_{\text{pref}}[i]$ means that patient p_i has that GP as their preferred GP, and an edge $D_{\text{cur}}[i] \to p_i$ means that the GP currently has p_i as one of their patients. Edges always go between a patient and a GP, never between two patients or two GPs, so the graph is bipartite with patients on one side and GPs on the other. A directed cycle in this graph alternates between patient and GP nodes and corresponds to a valid exchange, each patient in the cycle moves to their preferred GP, and each GP loses one patient and gains one. This representation is used by *Greedy DFS* and by the TTC of Huitfeldt et al., both of which search for cycles directly among the patients and GPs.

4.1.2. GP collapsed graph

$$\begin{gathered} G = (V, A), \quad V = D \\ A = \left\{ (a, b) \mid \exists i \in I \ \text{ s.t. } \ D_{\text{cur}}[i] = a \wedge D_{\text{pref}}[i] = b \right\} \\ u(a, b) = \left| \left\{ i \in I \mid D_{\text{cur}}[i] = a \wedge D_{\text{pref}}[i] = b \right\} \right| \\ c(a, b) = -1 \end{gathered} \tag{13}$$

In the *GP collapsed graph* only the GPs are nodes. An edge $d_a \to d_b$ means that there exists at least one patient who currently has GP d_a and has GP d_b as their preferred GP. All patients wanting the same switch are collapsed into one edge, the capacity $u(a, b)$ counts how many

they are, and the cost of every edge is -1. What we gain with this representation is a small graph, the number of nodes is the number of GPs and identical requests share one edge. What we lose is the identity of the individual patients, the graph only knows how many patients want each switch, not which ones. This is exactly the information needed for the cardinality ordering, and this representation is used by *Cycle Cancelling for cardinality*.

4.1.3. GP multigraph

$$
\begin{aligned}
G &= (V, A), \quad V = D \\
A &= \left\{ \left(D_{\text{cur}}[i], D_{\text{pref}}[i], i \right) \mid i \in I \right\} \\
u(a, b, i) &= 1
\end{aligned}
\tag{14}
$$

The *GP multigraph* is much like the *GP collapsed graph*, but instead of collapsing all equal preferences we make each preference into its own edge. This makes the graph a multigraph, as two patients wanting the same switch give two parallel edges. Each edge then represents exactly one patient, identified by the index i, and has capacity one. Compared to the *GP collapsed graph* we get more edges, but we keep the identity of each patient, which the priority based orderings need. For the *Cycle Cancelling for utility* algorithm we additionally give each edge a cost, $c(a, b, i) = -R(p_i)$, encoding the priority of the patient in the edge weight. The *Cycle Cancelling for strict priority* algorithm uses the same graph but without costs.

4.2. Huitfeldt et al. TTC

To have a fair baseline to compare against we also implemented the TTC of Huitfeldt et al. [4], described in Section 2. Our implementation is a direct port of the Python code from their replication package to Rust, keeping the algorithm logic unchanged. The replication package is not publicly available at the time of writing, the authors kindly shared it with us directly. Porting it to Rust means all algorithms in our experiments run in the same language, so runtime comparisons are not skewed by the choice of language.

Two adaptations were needed to run it in our setting. First, their code expects priorities where a lower number means higher priority, as their priority is a position on a waitlist. In our setting a higher priority number means higher priority, so we reverse the order before passing patients to the algorithm. Second, their implementation first runs a waitlist algorithm that assigns patients to GPs with free capacity before running TTC. In our setting every GP is at full capacity, so this step never does anything and we omit it. The TTC itself is unchanged, GPs rank their current panel members in a fixed arbitrary order followed by the waitlisted patients in priority order, exactly as in their code.

4.3. Greedy DFS

The motivation behind *Greedy DFS* is to have a fast and simple heuristic that still respects priorities. It is inspired by the TTC of Huitfeldt et al., which preserves the TTC properties at each round by restricting each GP node to a single outgoing edge [4]. This restriction means their algorithm can miss exchanges, a cycle may exist through a GP's panel without going through the single patient the GP currently points to. We relax this constraint, allowing each GP to keep outgoing edges to all of its current patients at the same time, and resolve ties by patient priority. This lets the algorithm find cycles that the TTC structure misses, while still

favoring high priority patients. As we will see, the greedy choices do not guarantee an optimal solution, so *Greedy DFS* is a heuristic, and it serves as a fast point of comparison for the exact algorithms.

The algorithm runs on the *Patient and GP graph* defined in Section 4.1.1. Recall that a directed cycle in this graph corresponds to a valid exchange. The idea of the algorithm is simple, it goes through the patients in decreasing priority order, and for each patient it runs a depth first search that tries to find a cycle through that patient. If a cycle is found it is resolved immediately, every patient on the cycle moves to their preferred GP and is removed from the graph. When the search reaches a GP node with several current patients, it explores the highest priority patient first.

GreedyDFS(G, R)

```
resolved = []
sorted = patients sorted by R in decreasing order
for each p ∈ sorted do
    cycle = dfsFindCycle(G, R, p)
    if cycle ≠ none then
        for each q ∈ cycle do
            resolved.push(q)
        end
    end
end
return resolved
```

The depth first search dfsFindCycle(G, R, p) starts at patient p and follows edges through the graph. At a patient node it follows the one edge to that patient's preferred GP. At a GP node it has several edges to choose from, one to each current patient, and it tries them in decreasing priority order. The search returns a cycle if it reaches p again, and returns none if no such cycle exists. When a cycle is found, every patient in it is added to the resolved set and is not considered again.

The greedy rule makes sure high priority patients are preferred when forming cycles, but it does not guarantee an optimal solution. The following example shows how the greedy choice can be locally motivated but globally suboptimal.

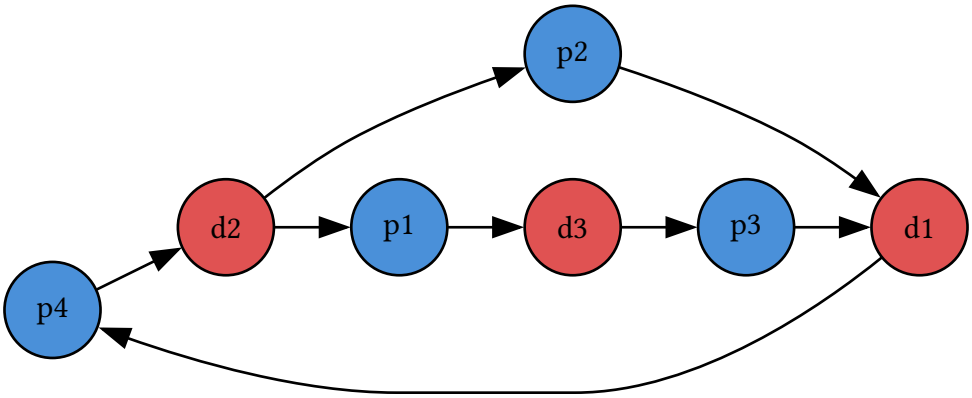

Figure 7: Example graph G where Greedy DFS would choose suboptimal solution. px means patient x and has priority x.

In Figure 7 each patient p_x has priority x. The search begins at p_4, the highest priority patient, and follows the edge to d_2. At d_2 it chooses p_2 over p_1 since $R(p_2) > R(p_1)$. The resulting cycle $p_4 \to d_2 \to p_2 \to d_1 \to p_4$ is committed, which leaves p_1 and p_3 unsatisfied with no further cycles remaining.

Had the search chosen p_1 at d_2 instead, it would have found the longer cycle $p_4 \to d_2 \to p_1 \to d_3 \to p_3 \to d_1 \to p_4$, which satisfies three patients. The greedy solution $\{p_4, p_2\}$ is suboptimal under both orderings, the alternative $\{p_4, p_1, p_3\}$ contains more patients, so it is greater under $\succ_{\text{size}}$, and it contains p_3 which outranks p_2, so it is also greater under $\succ_{\text{lex}}$. The greedy choice at d_2 was locally motivated by priority but globally suboptimal. This motivates the exact algorithms in the following sections.

4.3.1. Runtime

In the worst case a single DFS can visit every GP node and every patient edge before finding a cycle or giving up, costing $O(|D| + |P|)$ time. We start one DFS per switching patient, so the total runtime is $O(|P|\ (|D| + |P|))$ in the worst case. In practice the algorithm is much faster than this bound suggests, for two reasons. First, when a DFS finds a cycle, every patient on the cycle is resolved and removed, so successful searches pay for several patients at once. Second, when a DFS exhausts the options of a patient without finding a cycle, that patient cannot be part of any cycle in the current graph, we mark such patients as stuck and skip them in all later searches the same day. Each day consists of one DFS per unresolved switching patient, so without this pruning a hopeless patient could be re-explored by every one of these searches. With it, each failed patient is explored at most once per day. The marks are cleared when the graph changes, that is, on the next day of the simulation.

4.4. Cycle cancelling for cardinality

This algorithm finds an optimal solution under $\succ_{\text{size}}$, that is, a solution resolving the largest possible number of patients. The motivation for this ordering is the system view, every resolved patient is one person leaving the waitlist, so resolving as many as possible each day is the most direct way of keeping the waitlists small. The drawback is that all patients count equally, the algorithm has no notion of who has waited longest, so individual patients may be passed over day after day.

We can state this variant of the GP allocation problem as a minimum cost circulation problem. We use the *GP collapsed graph* defined in Section 4.1.2, where every edge has cost -1 and the capacity counts the patients wanting that switch. Recall from Section 2.3 that with all costs -1, a circulation with minimum cost is equivalent to a circulation with the most flow, and every unit of flow here is one resolved patient. So we find a minimum cost circulation f^* using *Cycle Cancelling*, and then read the solution off the flow, for each edge carrying k units of flow we resolve k of the patients wanting that switch.

CycleCancellingCardinality(G, P)

```
1  resolved = ∅
2  f = zero circulation on G
3  f* = CycleCancelling(G, f)
4  for each edge (v, w) ∈ G do
5  |  if f*(v, w) > 0 then
6  |  |  let S = {p ∈ P | D_cur[p] = v ∧ D_pref[p] = w}
7  |  |  pick any f*(v, w) patients from S and add them to resolved
8  |  end
9  end
10 return resolved
```

Because we choose edges based only on the capacity, e.g. the number of patients wanting that switch, we do not know which k patients to actually resolve when we find out that k patients can switch along an edge. This choice is arbitrary since all patients in this representation are equally important and it does not affect the cardinality of the solution. While we could choose random patients, the better choice might be to choose the k patients who have the highest priority among those who want the swap.

4.4.1. Runtime

The running time of *Cycle Cancelling* as mentioned is $O(nm^2CU)$ making the algorithm pseudo polynomial based on C, the maximum cost, and U, the maximum capacity. In the *GP collapsed graph* the maximum cost, C, is one, while the maximum capacity, U, is at most the number of patients. So the running time of *Cycle Cancelling for cardinality* is polynomial in terms of the input G, P as the running time is $O(nm^2\ |P|)$.

Because of the *Negative Cycle Optimality Theorem* the *Cycle Cancelling* algorithm will not finish until there are no more negative cycles in the residual graph. Since the cost of each edge in the original graph is -1 and *Cycle Cancelling* minimizes the total cost, the algorithm finds the optimal solution under $\underset{\text{size}}{\succ}$.

4.5. Cycle Cancelling for utility

This algorithm finds an optimal solution under $\underset{\text{util}}{\succ}$. The motivation for this ordering is to sit between the two extremes. We want to help patients who have waited a long time, but unlike the strict priority case, a single patient with a long wait should not always outweigh a group

of patients with shorter waits. Under $\underset{\text{util}}{\succ}$ every patient counts, weighted by their priority, and the base k of the priority function from Section 3 tunes how strongly long waits are favored.

The construction follows the same idea as for cardinality, but now the cost of an edge must carry the priority of its patient, so identical requests can no longer share one edge. We use the *GP multigraph* defined in Section 4.1.3, with the priority costs. Each patient i is their own edge in G with capacity one and cost $-R(p_i)$. A circulation then picks a feasible set of patients, and its cost is the negative of the total utility of that set, so a minimum cost circulation is an optimal solution under $\underset{\text{util}}{\succ}$. As before we find a minimum cost circulation with *Cycle Cancelling*, and every edge carrying flow corresponds to a resolved patient.

CycleCancellingUtility(G, P)

```
resolved = ∅
f = zero circulation on G
f* = CycleCancelling(G, f)
for each edge (v, w, i) ∈ G do
    if f*(v, w, i) > 0 then
        Add P[i] to resolved
    end
end
return resolved
```

4.5.1. Runtime

The running time differs from the cardinality case because the graph parameters are different. The maximum capacity U is now 1, since every patient is its own capacity-1 edge, while the maximum cost C is $\max R(p)$. The running time thus becomes $O(nm^2 \max R(p))$. This is polynomial as long as the priorities are polynomially bounded in the input size.

For the strict lexicographic case, the priority function is $R(p_i) = 2^i$, which grows exponentially in the number of patients. With this priority function C is exponential in n, so running this algorithm on it would no longer be polynomial. This is why we treat the strict lexicographic case separately.

There is also a practical limit on how large the priorities can get. In our implementation the costs are stored as 128 bit integers, and with $R(a) = k^{\text{days waiting}}$ the weights grow exponentially with waiting time. To keep the weights within bounds we divide the number of days by 10 before exponentiating, so patients are grouped into priority buckets of 10 days, and we cap the exponent so the weights never overflow. For the fastest growing base we use, $k = 1.9$, the cap is reached after 1010 days of waiting. Past this point all longer waits get the same weight and the ordering between them is lost. So in practice the utility algorithm with exponential priorities can only separate patients up to a bounded waiting time, this limits how long simulations we can run it on.

4.6. Cycle Cancelling for strict priority

The *Cycle Cancelling for strict priority* finds an optimal solution under $\succ_{\text{lex}}$. The motivation for this ordering is fairness towards those who have waited the longest. The algorithm never passes over a satisfiable patient in favor of anyone with lower priority, the patients who have waited the longest are helped first whenever helping them is possible at all. The cost of this guarantee is that a single high priority patient can block several lower priority patients, so we expect it to resolve fewer patients per day than the cardinality variant.

It uses the *GP multigraph* defined in Section 4.1.3, but without the priority costs. The two exact algorithms for priority thus solve their problems on the same graph, *Cycle Cancelling for utility* encodes priority in the edge weights while *Cycle Cancelling for strict priority* encodes it in the order patients are processed. It is based on *Cycle Cancelling*, but modifies it. We loop through the patients in descending order of priority. For each patient we check if their edge can be part of a cycle in the residual graph, if it can we push flow around that cycle and add the patient to the solution. If the patient's edge already carries flow, from being part of a cycle routed by an earlier, higher priority patient, the patient is simply added to the solution. Either way the patient's edge is then deleted from the graph, so that no later patient can undo the decision. The algorithm is as follows:

CycleCancellingStrictPriority(G, R, P)

```
1  resolved = ∅
2  f = zero circulation on G
3  sorted = P sorted by R in decreasing order
4  for each p ∈ sorted do
5  |  i = idx(p),    e = (D_cur[i], D_pref[i], i)
6  |  if f(e) = 1 then
7  |  |  Add p to resolved
8  |  else if ∃ directed path Q from D_pref[i] to D_cur[i] in G(f) then
9  |  |  Push one unit of flow on e and on every edge of Q
10 |  |  Add p to resolved
11 |  end
12 |  Delete e and its residual edges from G
13 end
14 return resolved
```

Note that when a cycle is cancelled in line 9, only patient p is added to the solution, not the other patients whose edges lie on the path Q. Those patients are not forgotten, their edges now carry flow, so they are added to the solution when their own turn comes in the loop, by the first branch in line 6. The flow on a routing edge can still be rerouted by a later patient, a patient is only finally committed at their own turn.

The final solution is the optimal solution under $\succ_{\text{lex}}$. To show why we first need a small observation about how the feasible solutions change as the algorithm runs.

Theorem 4.6.1 (Monotonicity of feasibility): Let F be the set of patient edges that currently carry flow, and let e be an edge not in F. If there is no circulation that has flow on every edge of F and on e, then there is also no such circulation after more patients have been committed.

Theorem 11: Monotonicity of feasibility

Proof: Committing a patient adds their edge to F and deletes the edge from the residual network. Both of these only add constraints, no new circulations become possible. So if no circulation containing $F \cup \{e\}$ exists now, none can exist later. □

Recall that a path from $D_{\text{pref}}[i]$ to $D_{\text{cur}}[i]$ in the residual network $G(f)$, together with the edge e, forms a cycle through e. By standard flow theory such a path exists if and only if there is a circulation that keeps flow on all committed edges and also has flow on e. So the search in the algorithm is an exact test of whether patient p can still be satisfied together with all higher priority patients that are already resolved.

Now consider the patients in the order the algorithm processes them. Because one high priority patient is "better" in terms of $\underset{\text{lex}}{\succ}$ than all patients with lesser priority combined, the optimal solution under $\underset{\text{lex}}{\succ}$ must contain patient p whenever p can be satisfied together with the already committed patients. This is exactly when the algorithm finds a cycle through e, so the algorithm resolves p if and only if the optimal solution contains p. By deleting e and its residual edge after committing, no later patient can reroute the flow off of e, so the decision for p is final. If the algorithm does not find a cycle through e, then by Theorem 11 no later step could have satisfied p either, so deleting e loses nothing. Repeating this argument for every patient in decreasing priority order gives that the returned solution agrees with the optimal solution under $\underset{\text{lex}}{\succ}$ at every position, so they are equal.

Note that with tied priorities the algorithm returns the optimal solution under its fixed tie-break order, as defined in Section 3. This is not necessarily the solution satisfying the most patients within each priority level, one tied patient committed early can block several others of the same priority. Which of the tied patients does the blocking is decided by the tie-break order, so two different tie-break orders can give solutions of different sizes, both equally valid under $\underset{\text{lex}}{\succ}$ with their respective orders.

4.6.1. Runtime

As we loop over all patients we have $|P|$ iterations. For each iteration we search for a path from $D_{\text{pref}}[i]$ to $D_{\text{cur}}[i]$ in the residual network. This is a single graph search, for instance a BFS, and takes $O(n + m)$ time. So the *Cycle Cancelling for strict priority* has a running time of $O((n + m)\ |P|)$ and is not dependent on the maximum priority as in the *Cycle Cancelling for utility*. Note that this is also faster than searching for negative cycles with Bellman-Ford, which would cost $O(nm)$ per patient. We can drop the costs entirely because the priority order is already enforced by the order we process patients in, the edge weights play no role in this algorithm.

4.7. Properties of the algorithms

In Section 2 we saw that TTC satisfies strategy proofness, individual rationality and Pareto efficiency. A natural question is which of these properties our algorithms keep.

Individual rationality holds for all our algorithms. A patient is either moved to the GP they reported as preferred, or stays at their current GP, no patient is ever moved somewhere they did not ask for. So no patient is made worse off by participating.

All our algorithms also return solutions that cannot be extended. Greedy DFS and the cycle cancelling algorithms only stop when no further cycle exists in their graph, so no additional patient can be satisfied without removing another from the solution. In this sense the solutions are Pareto efficient, we cannot improve one patient without worsening another. Note that this is a weaker statement than for classical TTC, where every agent has a full preference list and Pareto efficiency is over all preference profiles, in our setting a patient has only two outcomes, satisfied or not.

For strategy proofness we have to be more careful. In a single run, a patient reports only one preferred GP. Reporting anything other than the true preference can only result in being moved to a GP the patient does not want, or not being moved at all, so truthful reporting is a dominant strategy in the one shot setting. In the repeated setting the situation is less clear. Huitfeldt et al. [4] show that when the mechanism is run repeatedly and priorities depend on waiting time, TTC style mechanisms lose strategy proofness, patients can gain by timing their requests, and some patients can end up worse off than under a first come first served system. The same concerns apply to our algorithms, our priorities are based on days waited, so a patient could in principle time their request to affect their priority. We have not analyzed strategic behaviour in the repeated setting and leave it as future work.

5. Experiments and results

In this section we define the simulations we designed, algorithms to run on these simulations and metrics used to compare them. Then we present our hypotheses we had before running the experiments. Finally we present and discuss results from the simulations.

5.1. Experimental setup

Because of privacy and time constraints we could not use real data from Helfo. We then proceeded to use randomly generated data, while trying to keep relative sizes compared to real numbers in Norway. In the following sections we define how the data generation is done, our simulation model, exactly what algorithms we compared and the metrics we use to measure them.

5.1.1. Data generation

To generate the starting state for our simulation, we start by defining our parameters.

- The number of patients
- The number of doctors
- The percentage of patients starting on a waitlist
- The number of patients entering the waitlist each day
- The number of districts, districts model geography such that doctors and patients have a district they are bound to. Patients may change district when they get a doctor in another district.
- The percentage chance that a patient requests a doctor outside of their district.
- The number of days in the simulation
- The random seed

We start by creating the districts, assigning weights at random between 0.5 and 1.5 to each district. Then we assign doctors to each district depending on its weight, the higher the weight the more doctors a district gets, making sure each district has at least one doctor. After we assign doctors weights at random between 0.5 and 1.5. Based on these weights we distribute patients among doctors, the higher a doctors weight the more patients that doctor gets. We make sure each doctor gets at least one patient.

Lastly we pick the patients that start on a waitlist. For each patient picked we must choose a new preferred doctor. If the patient's district contains only one doctor then the patient chooses a preferred doctor randomly out of all the doctors except their own. Otherwise the patient chooses a random doctor withing their own district or outside depending on the parameter.

The parameters are chosen to match the proportions of the real Norwegian GP system. We could not run the full population size, as the exact algorithms based on Bellman-Ford scale with the number of GPs, and running all algorithms for the full simulation length at full scale was not feasible in the time we had. Therefore we scale the system down and keep the proportions matched to the real system instead. We simulate 100,000 patients and 102 GPs, giving about 980 patients per GP, matching the roughly 5.6 million people and 5720 GPs in Norway [2]. The initial waitlist is 5% of the patients, matching the 297,000 people waiting to switch GP in Norway as of December 2025 [2]. We use 6 districts, giving 17 GPs per district, matching the average of about 16 GPs per municipality in Norway. The probability that a

request crosses districts is set to 11%. As of December 2025, 33,500 of the roughly 297,000 people on a waitlist were waiting for a GP in another municipality or county than their own [2]. Note that this is the share of the outstanding waitlist and not the share of new requests, if cross district requests take longer to resolve they accumulate on the waitlist, so the true share of new requests is likely somewhat lower. We still consider this the best available estimate. The proportions are what drive the dynamics of the waitlists, so we keep these matched to the real system instead.

5.1.2. Simulation model

The simulation runs day by day. Each day consists of three steps. First, every patient on the waitlist has their priority and waiting time increased by one day. Then the algorithm being tested is run on the current state, every patient it resolves is moved to their preferred GP and removed from the waitlist. Finally, new switch requests are added, each from a patient chosen at random among those not already waiting, and with the preferred GP chosen using the district structure described above.

Each day 18 patients submit a new switch request. In 2025 there were about 370,000 self-chosen GP switches in Norway, of these about 218,000 went through a waitlist while the rest switched directly to GPs with open slots [2]. Our model has no free capacity, so we treat all voluntary switch demand as going through the exchange mechanism. Scaled to our population size this gives about 18 requests per day.

We record statistics each day, the size of the waitlist before and after the algorithm runs, the number of patients resolved, the waiting times of the resolved patients, and the wall-clock time of the algorithm itself. Waiting times are also recorded for the patients still on the waitlist when the simulation ends, so long waits are not hidden by never being resolved.

5.1.3. Algorithms compared

We compare five algorithms:

- *Huitfeldt TTC*, the existing mechanism we use as a baseline, described in Section 4.
- *Greedy DFS*.
- *Cycle Cancelling for cardinality*.
- *Cycle Cancelling for utility*, with the linear priority function $R(p) = a$ where a is days patient p has waited.
- *Cycle Cancelling for strict priority*.

Each of these we ran in a 100 year simulation, long enough for the waitlists to stabilize and to see the long term behaviour of each algorithm.

In addition we run five variations of *Cycle Cancelling for utility* with an exponential priority function $R(a) = k^{\lfloor a \,/\, 10 \rfloor}$, using the bases $k = 1.01, 1.05, 1.1, 1.5$ and 1.9. The days are divided into buckets of 10 to keep the weights within bounds, see Section 4. This means patients within the same 10 day bucket are weighted equally, the exponential variants order groups of patients rather than individuals. Recall from Section 4 that the exponential priorities have a practical limit on how long a patient can wait before the weights overflow, for $k = 1.9$ this limit is 1010 days. We therefore run the exponential variants in a smaller simulation of two years, since no patient can have waited longer than the simulation itself this keeps every

variant below the limit. The two year simulation also includes all five algorithms from the long run, so the exponential variants can be compared against them directly.

5.1.4. Metrics

To compare the algorithms we use different metrics to view the positive and negative sides of each one.

The first simple metric is the sum of patients resolved over the whole simulation. This metric on its own says little about how good an algorithm is, since an algorithm that resolves many patients can still leave some waiting a long time, but it lets us see whether the algorithms differ in how many patients they resolve at all, or only in which patients they resolve.

The most important metric is the size of the waitlist over time. This is what the system as a whole cares about, a good algorithm should keep the waitlist small and stable.

The waitlist size alone can however hide unfairness. An algorithm can keep the waitlist small while letting a few unlucky patients wait forever. We therefore also measure the waiting times of the resolved patients, the average, the 99th percentile and the maximum. The 99th percentile shows how the algorithm treats its least lucky patients without being dominated by a single outlier as the maximum is. Waiting times are also recorded for the patients still on the waitlist when the simulation ends, so long waits are not hidden by never being resolved.

We also measure the share of the waitlist that is waiting for a GP in another district. Cross district requests shoul be harder to resolve as fewer patients want to switch the other way, this metric shows how each algorithm treats these patients. Recall from Section 5.1.1 that 11% of new requests are cross district, if an algorithm treats them no worse than other requests their share of the waitlist should stay near 11%.

Finally we measure the wall-clock time each algorithm uses per day. An exact algorithm is of little practical use if it cannot keep up with the system it is meant to run in.

5.2. Results

In this section we present the results of running simulations with the algorithms mentioned in Section 5.1.3. We ran two simulations in total.

The first is our main simulation. It runs *Greedy DFS*, *Huitfeldt TTC*, *Cycle Cancelling for cardinality*, *Cycle Cancelling for strict priority* and *Cycle Cancelling for utility* with a linear priority function $R(p) = a$, where a is the number of days patient p has waited. It runs for a simulated 100 years, or 36500 days, with the parameters defined in Section 5.1.1 and Section 5.1.2.

The second simulation adds the exponential variants of *Cycle Cancelling for utility*, with bases $k = 1.01, 1.05, 1.1, 1.5$ and 1.9, alongside the five algorithms from the main simulation. With a base greater than one the priority function grows rapidly as a patient waits, so the weights would overflow on a long simulation. For $k = 1.9$ this happens after 1010 days, see Section 4. We therefore run this simulation for only two years or 730 days, short enough that no weight overflows. It lets us compare the exponential variants against the other algorithms directly.

5.2.1. Total resolved

Figure 8 shows the total number of resolved patients in the simulation of 100 years. Note that patients can be counted more than once as a patient can request to switch GP more than once.

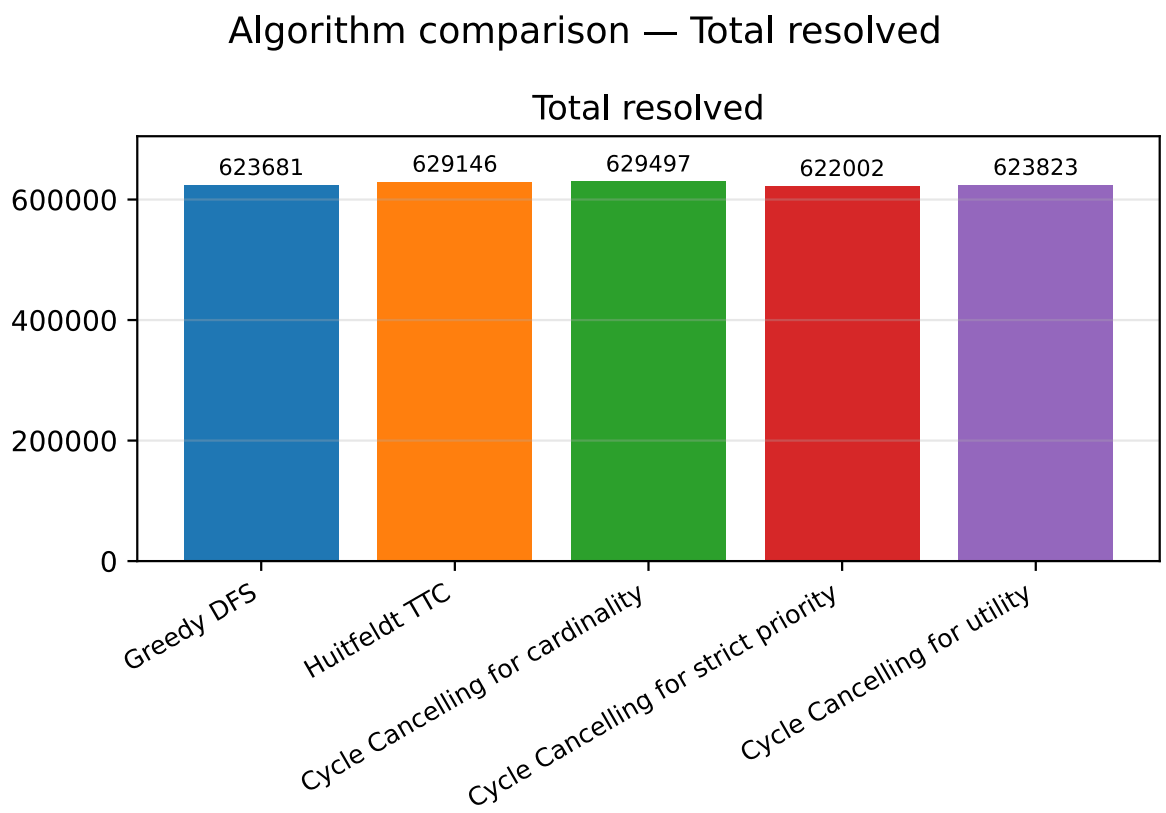

Figure 8: The total amount of resolved patients in each algorithm over the simulation of 100 years.

5.2.2. Waitlist size over time

Figure 9 and Figure 10 shows the size of the waitlists after the algorithm has ran each day. How the waitlist changes in the 100 year simulation is in Figure 9 and the two year simulation with exponential variants is in Figure 10. The lists in both simulations climb under every algorithm with *Cycle Cancelling for strict priority* ending highest and *Cycle Cancelling for cardinality* lowest.

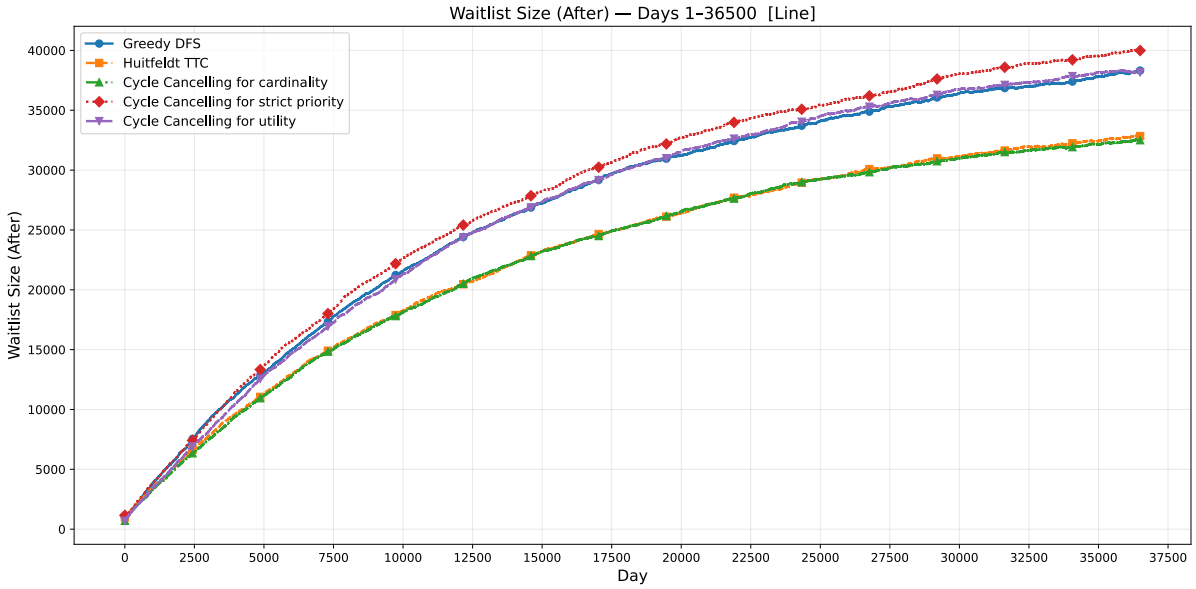

Figure 9: Waitlist size over time under each algorithm in the simulation, over 100 years at one tenth scale.

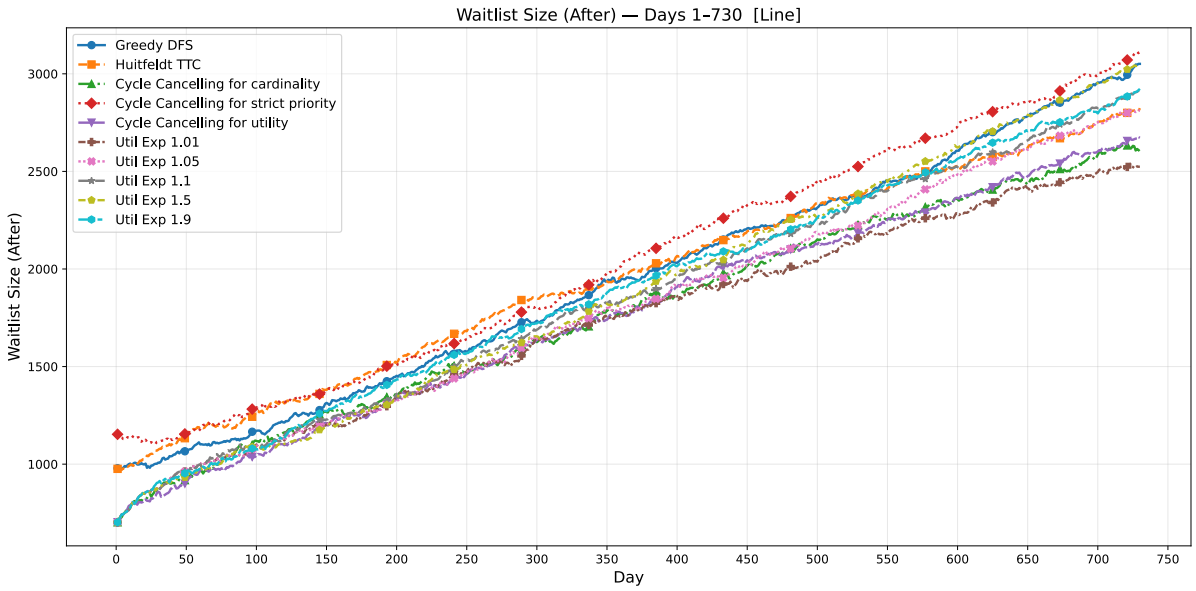

Figure 10: Waitlist size over time under each algorithm in the smaller simulation with exponential variants, over two year simulation.

5.2.3. Maximum waiting time

Figure 11 and Figure 12 shows how long patients wait before being resolved or never resolved. On the left we have how long on average the 99th percentile of patients that get resolved have to wait. By 99th percentile we ignore outliers that take very long and then can see how long most of the population have to wait. On the right we have the longest amount of time a patient waits before being resolved, incuding patients that are not resolved. If a patient is never resolved then the metric is equal to the number of days in the simulation. For Figure 11 this is 36500 days and for Figure 12 this is 730 days. *Cycle Cancelling for strict priority*, *Cycle Cancelling for utility* and *Greedy DFS* are all under the max bound while the others reach the full simulation length, meaning some patients are never resolved.

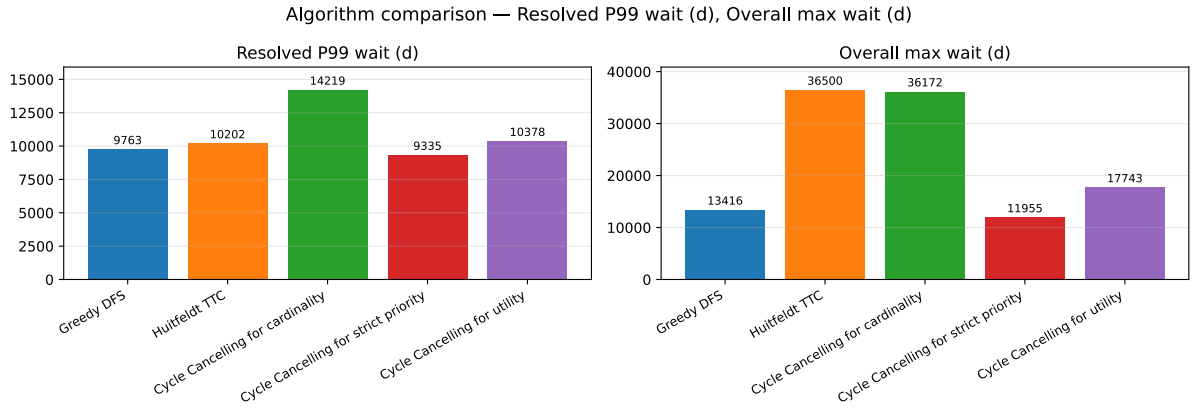

Figure 11: 99th percentile wait of resolved patients and overall maximum wait under each algorithm in the main simulation, over 100 year simulation. The maximum is taken over all patients, including those still waiting when the simulation ended.

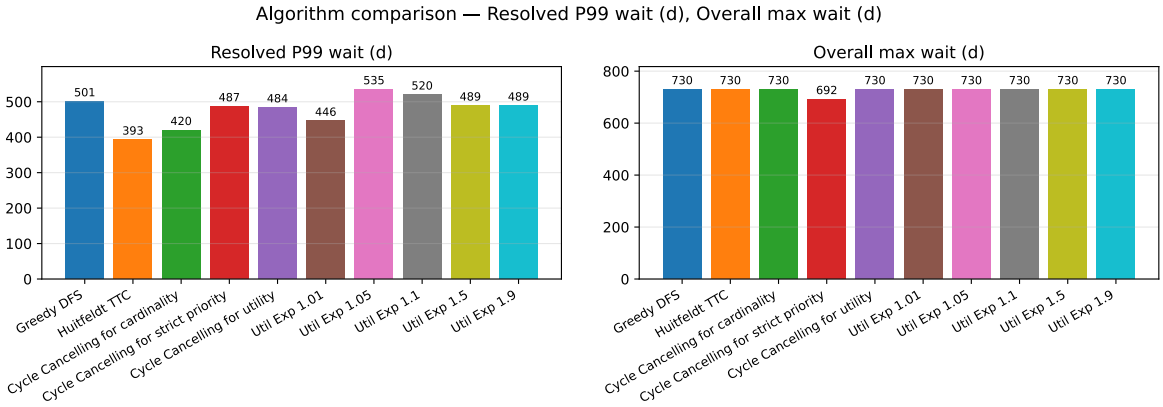

Figure 12: 99th percentile wait of resolved patients and overall maximum wait under each algorithm in the small simulation with exponential variants, over two year simulation. The maximum is taken over all patients, including those still waiting when the simulation ended.

5.2.4. Average waiting time

Figure 13 and Figure 14 show the average wait time for resolved patients. Figure 13 for the 100 year simulation and Figure 14 for the two year simulation. The average wait is lowest for *Cycle Cancelling for cardinality* and highest for *Cycle Cancelling for strict priority*, with the others in between.

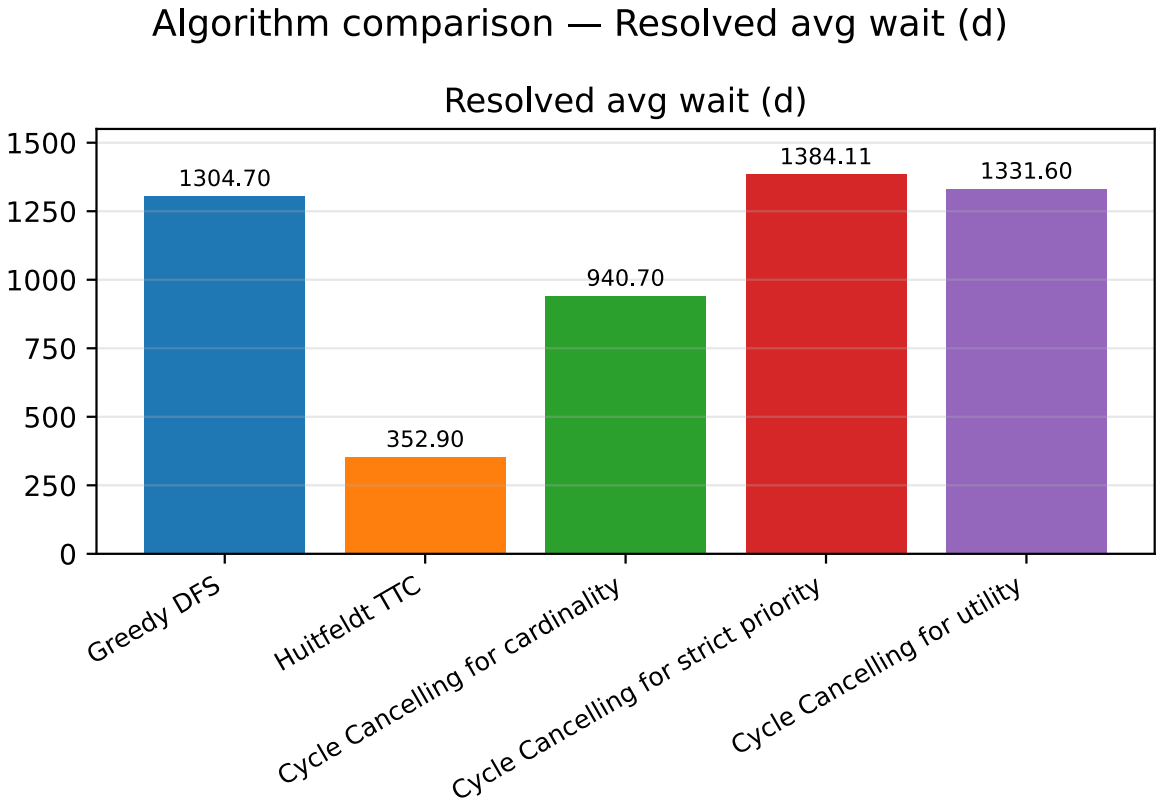

Figure 13: The average wait time for resolved patients in days. For 100 year simulation.

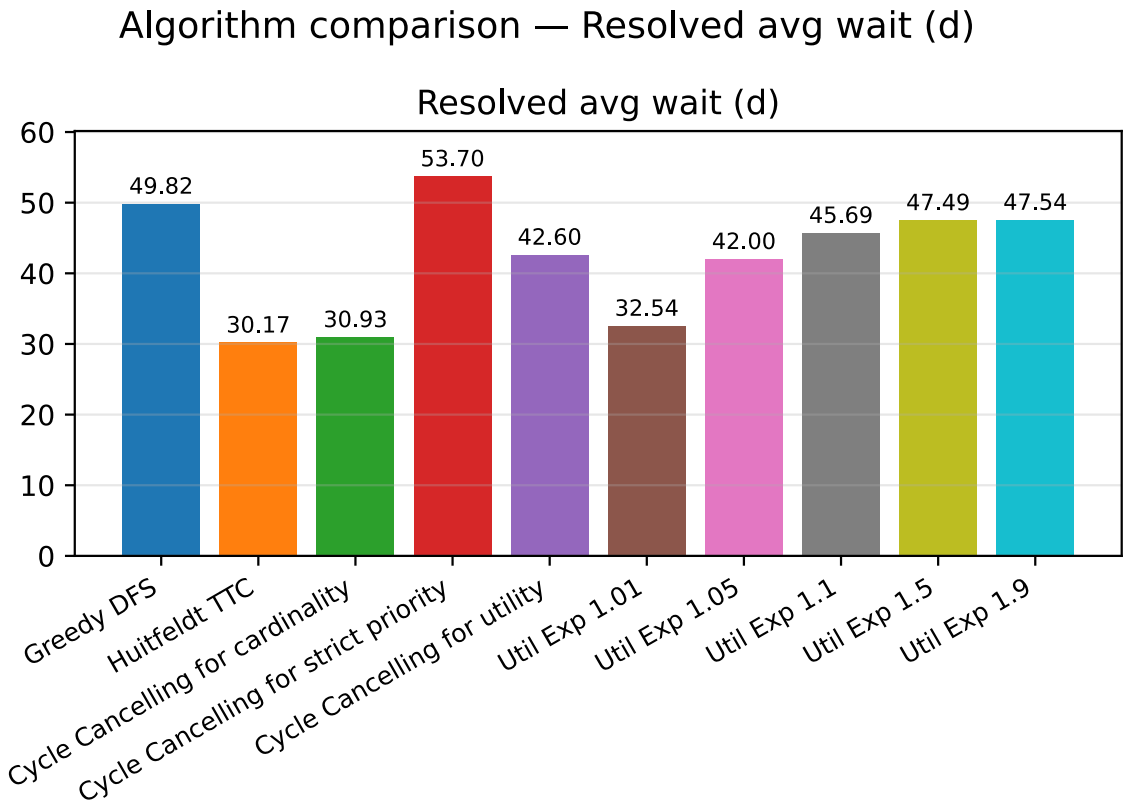

Figure 14: The average wait time for resolved patients in days. For the two year simulation with the exponential variants.

5.2.5. Runtime

Figure 15 and Figure 16 show how many milliseconds on average each algorithm takes per day.

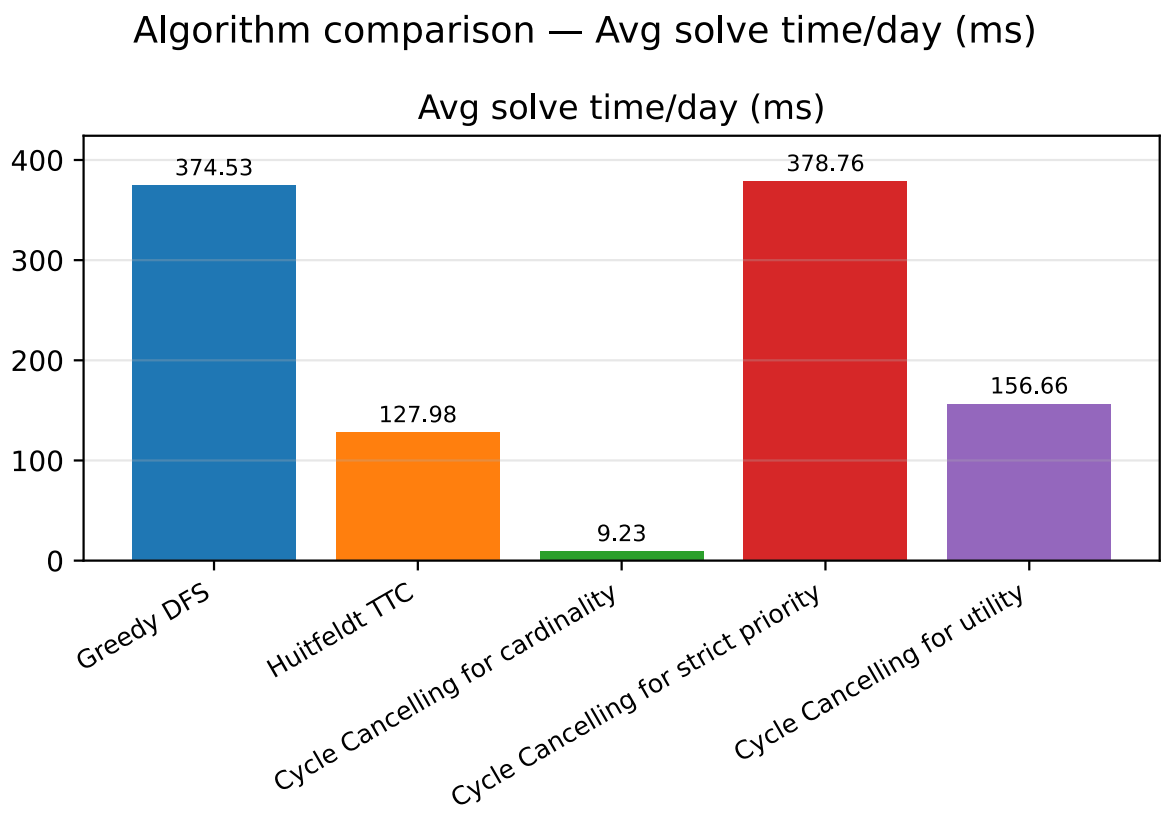

Figure 15: The average running time per day for each algorithm. For the 100 year simulation.

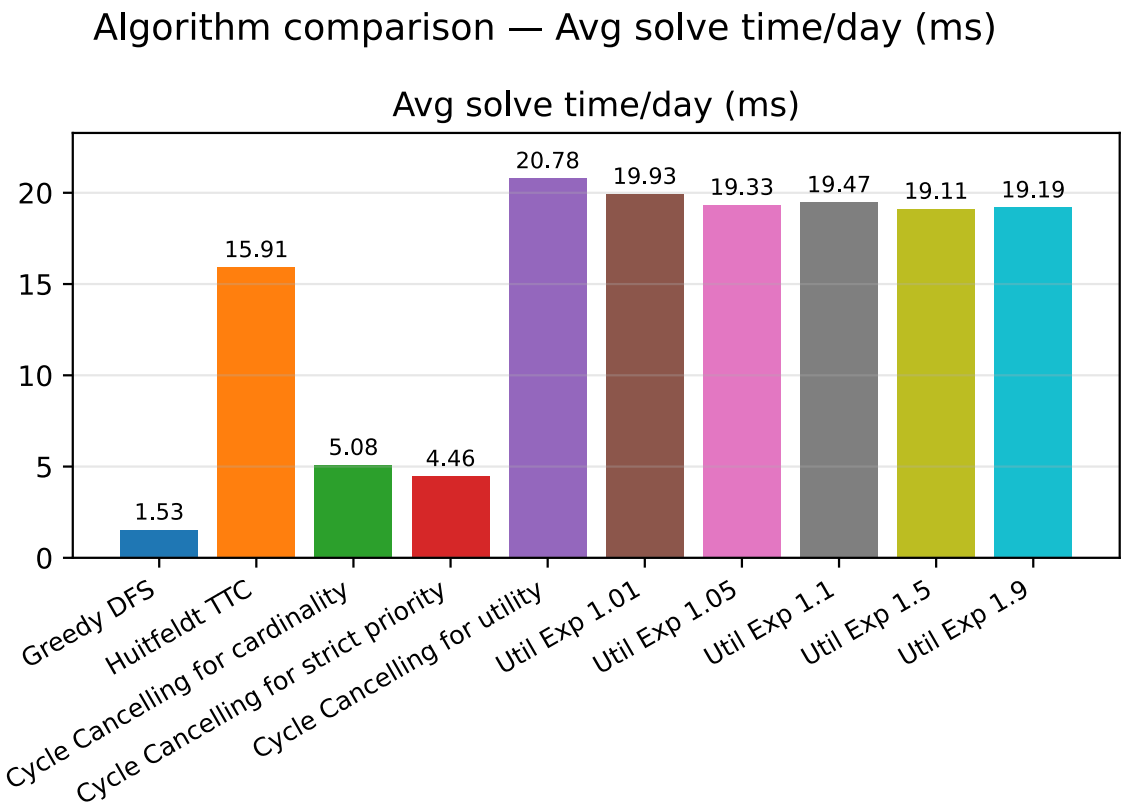

Figure 16: The average running time per day for each algorithm. In two year simulation with exponential variants.

5.3. Discussion

The clearest thing to say about the algorithms is that they all resolve almost the same number of patients. Figure 8 shows that over the whole simulation the most and the fewest differ by just over one percent, from 622002 resolved under *Cycle Cancelling for strict priority* to 629497 under *Cycle Cancelling for cardinality*. Since a patient who is resolved can submit a new switch request later and rejoin the waitlist, this is a count of switches carried out over the whole run and not of distinct people, so there is no fixed total that forces the algorithms together. That they still end up within about one percent of each other tells us that this metric is not what separates them. They all carry out close to the same number of switches, and the real difference is in which patients they serve. This is also what produces the differences in waiting time that we discuss below. An algorithm that resolves the same number of patients but always picks the ones who have waited the longest looks very different from one that picks whichever patients lead to the largest solutions.

The size of the waitlist is where this first shows up. In Figure 9 *Cycle Cancelling for cardinality* ends with the smallest waitlist, which is what we expected, since it resolves the largest number of patients each day. The result we did not expect is that *Huitfeldt TTC* follows it so closely, ending almost level with cardinality. We thought that the *Huitfeldt TTC* was too restrictive, but it nearly matches the algorithm that resolves the most patients. Below these two, *Cycle Cancelling for utility* and *Greedy DFS* end with larger and almost equal waitlists, and *Cycle Cancelling for strict priority* ends with the largest of all. That strict priority keeps the largest

list is expected, it spends its cycles on the requests that have waited the longest rather than on the requests that would let it resolve the most, so it clears its backlog more slowly.

The waiting times show why the size of the waitlist alone is misleading, and they reveal the central trade-off between the algorithms. The two algorithms with the smallest waitlists, *Huitfeldt TTC* and *Cycle Cancelling for cardinality*, also give the shortest waits for the typical request. In Figure 13 their average waits are by far the lowest, 352.9 days for *Huitfeldt TTC* and 940.7 days for *Cycle Cancelling for cardinality*, against more than 1300 days for each of the other three. But the same two algorithms have the longest waits for the unluckiest requests. In the maximum wait in Figure 11 they reach 36500 and 36172 days, essentially the entire length of the simulation, while the other three keep their maximum well below this. So the two algorithms that serve the typical request the fastest are exactly the ones that let a few requests wait almost the whole simulation, and the algorithms that protect those few requests do so by making the typical request wait longer. This trade-off runs through all of our results.

The 99th percentile wait in Figure 11 shows the same split from a less extreme angle. Here *Cycle Cancelling for cardinality* is the worst at 14219 days and *Cycle Cancelling for strict priority* is the best at 9335 days, with the others in between. So even short of the absolute maximum, cardinality lets its slowest requests wait the longest, while strict priority keeps them the most contained.

Cycle Cancelling for strict priority is the clearest example of the protective end of this trade-off. It has the worst average wait, at 1384.1 days in Figure 13, but the best 99th percentile wait at 9335 days and the best maximum wait at 11955 days in Figure 11. This follows from the ordering. Because a request with higher priority always counts for more than any number of requests with lower priority, and our priority grows with the days a request has waited, the algorithm serves the longest waiting requests first whenever a cycle through them exists. So a request that keeps waiting climbs in priority until the algorithm resolves it, and no single request sits unresolved far longer than the rest, even though a patient resolved on one request may submit another later. The price is that the algorithm spends its cycles on requests that have already waited a long time rather than on the requests that would let it resolve the most, so the typical request waits longer.

Greedy DFS and *Cycle Cancelling for utility* sit between the two ends on the maximum wait. Neither reaches the full length of the simulation, with maximum waits of 13416 and 17743 days in Figure 11, so unlike *Huitfeldt TTC* they do not let any request wait the whole simulation. *Greedy DFS* does this because it goes through the requests in order of priority, like *Cycle Cancelling for strict priority*, so the longest waiting ones are served whenever a cycle through them exists, but its greedy and sometimes suboptimal choices give it a worse 99th percentile and maximum wait than *Cycle Cancelling for strict priority*, at 9763 and 13416 days against 9335 and 11955 in Figure 11. *Cycle Cancelling for utility* also keeps the maximum wait bounded, since with our linear priority a request that keeps waiting gains weight until it is resolved. This is worth noting, since one might expect the linear utility ordering to let a single long waiting request be outweighed by a group of requests with shorter waits and so be left behind, but over a long simulation the weight of a waiting request grows without bound and eventually forces its resolution. On the typical request, though, *Cycle Cancelling for utility* behaves more

like strict priority than like cardinality, with an average wait of 1331.6 days, because over a hundred years the linearly growing weights come to favour long waiting requests strongly.

It is worth asking which requests are the ones left waiting almost the whole simulation under *Cycle Cancelling for cardinality* and *Huitfeldt TTC*. A request can only be resolved when the GP it prefers can reach the GP it currently holds through a chain of other patients who want to switch, that is, when the request lies on a cycle. Two situations make this unlikely. The first is when few or no other patients want the request's current GP, so there is no one to take over that slot and close a cycle through it. The second is when the request prefers a GP that few patients are trying to leave, whether because its panel is small or because its patients are mostly content, so a chain can rarely continue past that GP. In both cases the request lies on few cycles, or none, and so has little chance of being picked when cardinality resolves the requests that lead to the largest solutions. Since the same patient can submit such a request again and again, these are requests that stay stuck across the whole run rather than a one off piece of bad luck, and they are a property of individual GPs and not of geography. One might expect them to be requests for a GP in another district, since fewer patients want to switch in the opposite direction across a district boundary, but this is not the case, as shown in Figure 17.

Runtime is the last thing that separates the algorithms, and here the order between them changes with the size of the system. In the main simulation in Figure 15 *Cycle Cancelling for cardinality* is by far the fastest, at 9.2 milliseconds per simulated day against 128 to 379 milliseconds for the others. The reason is that it works on the collapsed graph, where the nodes are the GPs and all patients wanting the same switch share one edge, so its running time grows with the number of GPs rather than the number of patients. The other algorithms keep each patient separate, so their running time grows with the number of patients, which is far larger, and *Cycle Cancelling for strict priority* and *Greedy DFS*, which run a separate graph search for each patient, are the slowest at 378.8 and 374.5 milliseconds per day. In the smaller two year simulation in Figure 16 the order changes, *Greedy DFS* is the fastest at 1.5 milliseconds per day and the utility variants are the slowest at around 20, since with far fewer patients the per patient searches become cheap while the heavier work the utility solver does on its larger graph does not shrink as much.

The exponential variants of *Cycle Cancelling for utility* confirm that the utility ordering is a single spectrum with cardinality and lexicographic priority at its ends, as we argued in Section 3. This is clearest in the average wait in Figure 14, where the variants rise steadily with the base, from 32.5 days at base 1.01, almost the same as the 30.9 days of *Cycle Cancelling for cardinality*, up to 47.5 days at bases 1.5 and 1.9, approaching the 53.7 days of *Cycle Cancelling for strict priority*. At a base near one the utility ordering is almost the same as maximising the number of patients resolved, which is what the theory predicts, and as the base grows it favours long waiting requests more strongly. The waitlist size in Figure 10 does not order the variants as cleanly. The lowest base variant still ends with one of the smallest waitlists and the higher base variants with larger ones, but in between the variants are interleaved with each other and with the other algorithms rather than lining up by base. We expected the waitlist size to follow the base in the same way the average wait does, and it does not. The likely reason is that the waitlist size depends on which requests happened to be resolved early and in

what order, and over only 730 days, with the lists still small and climbing, this leaves enough day to day variation to scramble variants that resolve almost the same number of requests.

There is one thing that has to be remembered when reading all of these results, which is that our simulations assume every GP is at full capacity. As we noted in Section 1 not all GPs in Norway are full, and in reality slots open all the time as patients move away, pass away or as the panel caps change. In our closed model a request can only ever be resolved by an exchange, so the requests that lie on few cycles, the same ones that give the long maximum waits above, are resolved slowly if at all and build up on the list over time. This is why the waitlists in Figure 9 keep climbing even after a hundred years. In reality a patient whose request lies on few cycles would be served the next time a nearby slot opens up rather than waiting indefinitely, which is part of why the current system works and why no one waits forever in practice. A real deployment of any of our algorithms would, like the mechanism of Huitfeldt et al. [4], first run a waitlist step that fills the open slots and only then run the exchange.

For this reason the climbing waitlists should not be read as a prediction about Norway, they are a property of the closed model and not of the algorithms. What the experiments do show is that the exchange resolves requests that would otherwise keep waiting, so running one of these algorithms on top of the existing waitlist clearing should make the lists shorter. How much shorter, under the real rates at which patients arrive and leave, our model cannot say. What our experiments do establish is the relative ordering of the algorithms, and this is what the chapter rests on. Our results still come with limitations. We use randomly generated data instead of real data from Helfo, although we keep the proportions matched to the real Norwegian numbers. We have also not analysed strategic behaviour in the repeated setting, which as Huitfeldt et al. [4] show can break the properties that hold in a single run, since our priorities depend on the days a request has waited and a patient could in principle time a request to change their priority. Finally the requests left waiting almost the whole simulation under *Cycle Cancelling for cardinality* and *Huitfeldt TTC* are a real problem that a deployment would have to handle, for instance by combining the high throughput of cardinality with a rule that forces the resolution of any request that has waited beyond some limit.

6. Conclusion

Lastly we try to summarize our work and point out future work and how this thesis can be extended.

6.1. Summary

This thesis studied how the waiting lists in the Norwegian GP system can be reduced by resolving waiting cycles, cases where a group of patients each want a GP that another patient in the group currently holds. As of December 2025 about 300000 people were waiting to switch GP in Norway [2], and because almost all GP lists are full these patients can only be served by exchanging slots among themselves. We formalised this as the problem of finding vertex disjoint cycles in a directed graph of patients and GPs, and defined three notions of an optimal set of switches, resolving as many patients as possible, serving the highest priority patients first in lexicographic order, and a utility ordering that weighs every patient by their priority and that contains the other two as its endpoints. We implemented an exact algorithm based on cycle cancelling for each ordering, a fast *Greedy DFS* heuristic, and a port of the TTC mechanism of Huitfeldt et al. [4] as a baseline, and compared them in a simulation of the Norwegian system. The main finding is that the choice of objective has almost no effect on how many patients are resolved, the algorithms differ by only about one percent, but a large effect on which patients are resolved and therefore on how long they wait. *Cycle Cancelling for cardinality* keeps the smallest waitlist and the lowest typical wait, and is also the fastest of our algorithms, but lets a small number of requests wait almost indefinitely. *Cycle Cancelling for strict priority* makes the opposite trade, it gives the typical request the longest wait but keeps the worst case the most contained, since a request that keeps waiting climbs in priority until it is resolved. The utility ordering sits between these two, and its base acts as a single dial between resolving the most patients and protecting those who have waited longest. We did not expect the TTC baseline to keep pace with *Cycle Cancelling for cardinality* on the size of the waitlist, but it does, despite its more restrictive cycle structure.

6.2. Future Work

Several directions remain open. The most immediate is to run the algorithms on real data from Helfo rather than the randomly generated data we use, which would test whether the proportions we assumed hold in practice and would turn our relative comparison of the algorithms into a quantitative estimate of how much they could shorten the waitlists in Norway. Related to this, our simulations assume a closed system in which every GP is at full capacity and slots open only through exchange, which is why the waitlists keep climbing over a hundred years. A more realistic model would let slots open as patients move away, pass away or as panel capcities change, and would run a waitlist step that fills these open slots before the exchange, as the mechanism of Huitfeldt et al. [4] does.

A second direction concerns the requests left waiting almost the whole simulation under *Cycle Cancelling for cardinality* and *Huitfeldt TTC*. These are requests that lie on few cycles, and since the same patient can submit them again and again they stay stuck across the whole run. A practical mechanism might combine the high throughput of *Cycle Cancelling for cardinality* with a rule that forces the resolution of any request that has waited beyond a fixed limit, keeping the waitlist small while still bounding the worst case.

A third direction is strategic behaviour in the repeated setting. Our priorities grow with the days a request has waited, so a patient could in principle time a request to gain priority, and Huitfeldt et al. [4] show that TTC style mechanisms can lose strategy proofness and leave some patients worse off than first come first served when run repeatedly. We have not analysed this for our algorithms, and a formal treatment is left as future work.

Finally there is room to make the exact algorithms faster and richer. The utility algorithm is limited by its dependence on the size of the priority weights, and a strongly polynomial method such as the minimum mean cycle cancelling of Goldberg and Tarjan [11] could remove this limit. One could also restrict the cycle length, as is done in kidney exchange so that all swaps can be carried out at once, or let GPs express preferences over patients, making the problem two sided.

Appendix

A Cross district request outcomes

Since none of the algorithms are aware of districts, the district structure we added for realism could in principle leave cross district requests, where fewer patients want to switch in the opposite direction, systematically worse off. The figure below shows it does not.

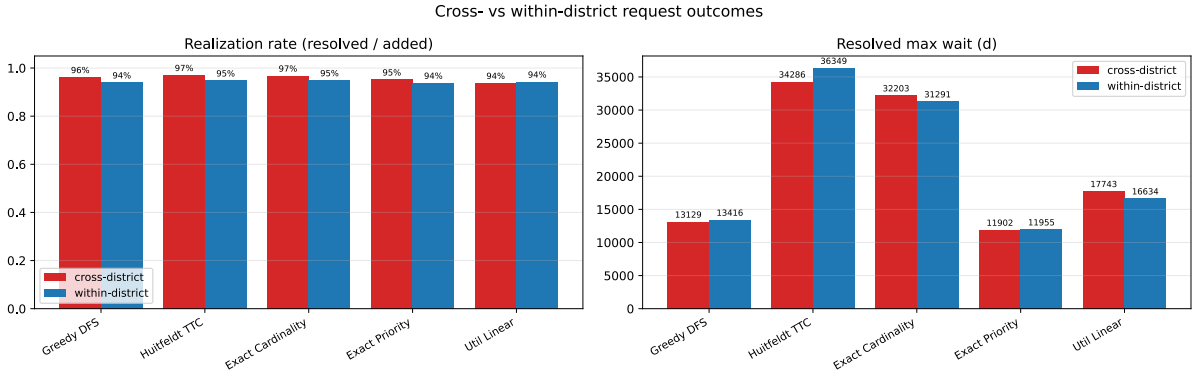

Figure 17: Cross district and within district request outcomes under each algorithm, in the main 100 year simulation. Left: the realization rate, the share of added requests eventually resolved. Right: the longest wait among resolved patients.

B Source Code Repository

All code used in this thesis can be found in the following repository: https://github.com/Reinsdyret/master°

C Statement on the Use of AI Tools

Claude was used throughout the thesis to provide structure, grammar correction, spelling fixes and discussing ideas with. Grammarly was also used for this same purpose. Claude Code was used as a programming assistant, but only when I knew exactly what i wanted done and how. It helped implement the python scripts to generate random data, plot results and other small scripts. In addition it helped implement the simulation and locate and fix bugs. All work these AI tools did i was aware of and I checked their work and made sure this is what i wanted. I am aware that I am responsible for all content of this master's thesis.

Bibliography

[1] Legelisten, "Med rett til å bytte – Ventetider for å bytte fastlege i Norge." Accessed: Apr. 08, 2026. [Online]. Available: https://legelisten.s3.eu-west-1.amazonaws.com/blog-files/Med+rett+til+%C3%A5+bytte+-+Ventetider+for+%C3%A5+bytte+fastlege+i+Norge.pdf

[2] Helfo, "Fastlegestatistikk." Accessed: Apr. 08, 2026. [Online]. Available: https://www.helfo.no/Fastlegeordninga/fastlegestatistikk

[3] Statistisk sentralbyrå, "Fastlegelister og fastlegekonsultasjoner, etter år og region (12005)." Accessed: Apr. 08, 2026. [Online]. Available: https://www.ssb.no/statbank/table/12005

[4] I. Huitfeldt, V. Marone, and D. C. Waldinger, "Designing Dynamic Reassignment Mechanisms: Evidence from GP Allocation," Working Paper 32458, May 2024. doi: 10.3386/w32458.

[5] L. Shapley and H. Scarf, "On cores and indivisibility," *Journal of Mathematical Economics*, vol. 1, no. 1, pp. 23–37, 1974, doi: https://doi.org/10.1016/0304-4068(74)90033-0.

[6] D. J. Abraham, A. Blum, and T. Sandholm, "Clearing algorithms for barter exchange markets: enabling nationwide kidney exchanges," pp. 295–304, 2007, doi: 10.1145/1250910.1250954.

[7] Wikipedia contributors, "Optimal kidney exchange — Wikipedia, The Free Encyclopedia." Accessed: May 05, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Optimal_kidney_exchange&oldid=1351916222

[8] D. Gale and L. S. Shapley, "College Admissions and the Stability of Marriage," *The American Mathematical Monthly*, vol. 69, no. 1, pp. 9–15, 1962, Accessed: May 06, 2026. [Online]. Available: http://www.jstor.org/stable/2312726

[9] A. E. Roth, "Incentive compatibility in a market with indivisible goods," *Economics Letters*, vol. 9, no. 2, pp. 127–132, 1982, doi: https://doi.org/10.1016/0165-1765(82)90003-9.

[10] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.

[11] A. V. Goldberg and R. E. Tarjan, "Finding minimum-cost circulations by canceling negative cycles," *J. ACM*, vol. 36, no. 4, pp. 873–886, Oct. 1989, doi: 10.1145/76359.76368.

Index of Figures